The Remote Support Program

Handover handbook, V2.5. Prepared 2026-10-28 from commit 8159021 (2026-10-28).

Start here

The Remote Support Program — handover pack, V2.5

This pack is for whoever takes over **The Remote Support Program** if Clint (the founder) can no longer look after it. It is written for a software development team, but it uses plain language so the business side can follow it too.

V2.5, prepared 28 October 2026, is **the final version before the product goes public on 1 November** — it replaces V2.4 (27 October 2026), and **document 15** lists the little that changed since, above all that **the guide (PDF) is now in the zip**. Document 14 lists what V2.4 added to V2.3 (16 October 2026): **three plans** (Full, Basic and Free), **more seats on request**, **job types**, **latest activity**, a technician asking for **outside help**, **exporting a company's data**, the fixes from an **outside security review**, **removing a client**, and the **website ready to go public**. Document 13 lists what is new since V2.2: **every program now keeps itself up to date** (customer computers silently, the console before its sign-in window), released to our own company first, then the pilots, then everybody, with a rollback if a new version does not come back. Document 12 lists what V2.2 added to V2.1: the live server moved to **PostgreSQL**, **PayFast** card payments, the **real website** (still hidden), **platform admins and a control dashboard**, **code signing** in use, **pilot companies**, and a dozen ticket and remote-control improvements. Document 11 lists what V2.1 added to V2 (above all **end-to-end encryption**), and document 10 what V2 added to V1 (2 October 2026), the biggest being a **free version** of the product. To refresh the pack later, run `tools\Make-HandoverPack.ps1` on the build PC.

What this product is, in three sentences

1. It is a **remote support tool**, like TeamViewer or AnyDesk. An IT technician can see and control a customer's Windows computer over the internet, with the customer's permission.
2. It is also a **job and ticketing system** for an IT business: customers, their machines, workshop bookings, quotes, invoicing stages, photos, and staff chat.
3. It is sold to **other IT companies** — and other businesses that look after other people's computers and jobs — as an online service. Each company gets its own branded copy (its own logo, colours and download links), and all of them run on one shared server at **https://theremotesupportprogram.com**. The **Basic Version for Remote Workers** (R50 a month since 2026-10-25) lets one person reach up to three of their own computers (more at R10 each, up to 20), for example the office PC from home, attended or unattended; the **Free Version** does the same for ONE computer, unattended, for nothing. Both can be upgraded in place.

Every part of it was written from scratch for this business. No remote-control engine was bought in or licensed, so there are no licence fees and nobody else's code to depend on.

The company that owns it is **The Remote Support Program (Pty) Ltd**, registration **2026/737434/07** (South Africa). Its first customer is **Anywhere IT**, Clint's own IT company.

What is in this zip

Folder or file	What it is
`00-START-HERE.md`	This page.
`The Remote Support Program - Guide.pdf`	**The guide**, 30 pages, for anybody thinking of using or investing in the product: what it does and what it costs, with pictures. Start here if you are not a developer.
`Handover documents\`	The plain-English documents listed below, as text files (`.md`) and as one web page you can open in any browser (`Handover handbook.html`).
`Source code\`	Every file of the product: the programs, the website, the installers, the scripts, the test tools and all the technical documents. This is a copy of the latest version.
`Source code with history\`	The same code with its **full history** — every change ever made, when and why. A developer restores it with one command (see the `HOW-TO-RESTORE.txt` inside it).

Unzip it near the top of a drive (for example `C:\Handover`). Some file paths inside are long, and Windows refuses paths over 260 characters if the zip is unpacked deep inside other folders.

Not in this zip, on purpose: passwords, encryption keys, the live database, and the server login key. Putting those on a USB stick would mean anyone holding the stick could take over every customer's computer. Document 08 lists every one of them, says where each lives, and explains how to hand them over safely.

Read these in order

#	Document	Who it is for	What it answers
01	What it is and how the pieces fit	Everyone	What the parts are and how they talk to each other
02	Every feature, and how it works	Everyone	What the product does, one feature at a time
03	How the security works	Everyone, then developers	How customers' machines and data are protected

#	Document	Who it is for	What it answers
04	Where it runs	Developers, the business	The server, the website address, email, backups, suppliers
05	Building and releasing	Developers	How to turn the code into programs and put them live
06	Map of the code	Developers	Which folder holds what
07	Where things stand	Everyone	What is finished, what is not yet tested, known faults, what was next
08	Keys, passwords and accounts	The business, then developers	Everything NOT on this stick and how to get it
09	Glossary	Everyone	What the words mean
10	What is new in V2	Anyone who read V1	Everything added or fixed between 2 and 6 October 2026
11	What is new in V2.1	Anyone who read V2	Everything added on 7 October 2026
12	What is new in V2.2	Anyone who read V2.1	Everything added between 8 and 15 October 2026
13	What is new in V2.3	Anyone who read V2.2	Automatic updates, added on 16 October 2026
14	What is new in V2.4	Anyone who read V2.3	Everything added between 17 and 27 October 2026
15	What is new in V2.5	Anyone who read V2.4	The final version before going live, 28 October 2026

After these, a developer should read the technical documents in `Source code\docs\`, starting with `HANDOVER.md` (the detailed engineering record, 173 numbered sections, about 12,500 lines), `ROADMAP-PARITY.md` (the build plan) and `NEXT-SESSION-FREE.md` (the free version, in Clint's words). `Source code\CLAUDE.md` is a short index of all of it.

If you only have one hour

1. Read documents 01, 02 and 07 (or, if you know V2.4, just document 15; if you know V2.3, 14 and 15).
2. Read the table in document 08 and find out who holds the password manager.

3. Check the live website opens: https://theremotesupportprogram.com
4. Do not change anything on the live server until a developer has read document 04 and the `docs/GO-LIVE.md` runbook in the source code. Customer computers depend on it.

The three things most likely to cause damage

1. **Losing the server's key folder (`data/keys`).** It unlocks every stored workshop device password and email password. The nightly backup contains it (and an office PC can fetch a copy every night — `deploy/backup/README.md` ; check it is switched on), but the backup itself is locked with a passphrase kept in `/etc/rsp-backup-key` on the server. Both must be kept safe — see 08.
2. **Changing the setting called** `SetApplicationName("RemoteSupportPlatform")` in the server's `Program.cs` . It looks harmless. Changing it makes every stored password unreadable.
3. **Turning off two-factor sign-in on the live server.** It is switched off only inside test tools. A technician's password, without the second factor, would be enough to reach every customer machine of that company.

A note on how this was built

The product was built between September and October 2026 by Clint working with an AI coding assistant (Claude, by Anthropic). Every change was recorded in the history and in `HANDOVER.md` , including what went wrong and why. The `tools\` folder holds about 200 small test programs that prove each feature works — **run the relevant one before and after changing anything.** That habit is the main reason the product works, and it is worth keeping.

01 — What it is and how the pieces fit

The people involved

It helps to keep four kinds of people apart, because the product treats them differently.

Who	Example	What they use
The platform owner	The Remote Support Program (Pty) Ltd — Clint	The **owner portal** on the website: approves companies, sets prices and seats, the platform's own email
An IT company (a "tenant")	Anywhere IT, or any reseller	Their own **portal** (website) and the **technician console** (a Windows program)
A technician or office worker at that IT company	A technician, a receptionist, a supervisor	The console, the portal, or both, depending on their role
A customer of that IT company	A dentist's practice, an accountant	A small **tray app** on their computer, a download page, sometimes a web page

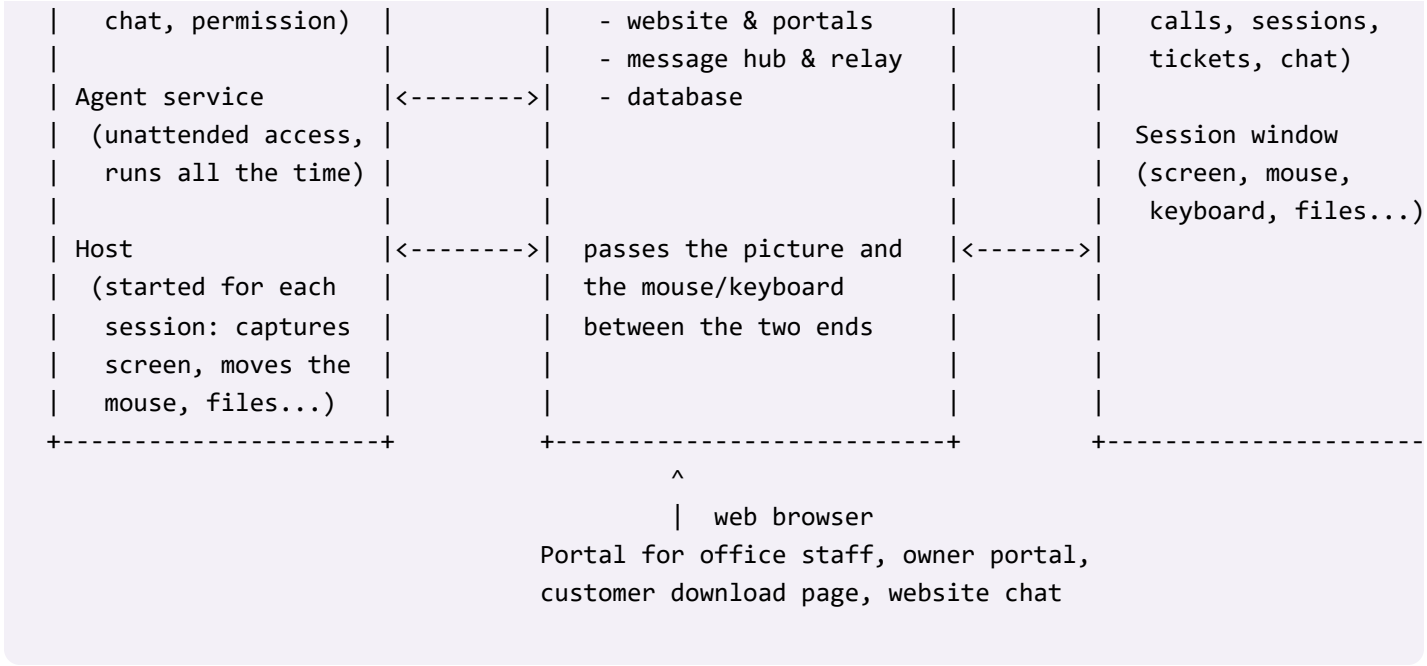
One IT company can never see another IT company's customers, machines or tickets.

Three versions (since 2026-10-25). The **full version** is everything in this pack, for IT companies supporting their customers. The **Basic version for remote workers** (R50 a month) is one person reaching **up to three of their own computers** (more at R10 each, up to 20) — typically the office PC or branch offices — unattended with a password, or attended with the person there asked to allow it; no tickets, no customers, no team. The **Free version** is the same for **one** computer, unattended only, with nothing to pay; it moves up to Basic or full, and Basic moves down to it. It runs on the same server and the same programs; a setting on the company (`Plan`) decides which rules apply, and the server enforces them. It can be **upgraded in place** to the full version. Document 02, Part H describes it.

The pieces

There are five programs and one website. They are all written in **C#** on **Microsoft .NET 10** (moved from .NET 8 on 2026-10-14).





On the customer's computer (three programs)

- **The tray app** (`RemoteSupport.Client`). The small icon by the clock. The customer uses it to ask for help, chat, choose whether unattended access is allowed, set an unattended password, let somebody outside their IT company in with a code, and **uninstall** the whole product. It also asks the customer's permission before a technician connects. **A free-version computer has no tray app** — the same file is used only to install and set the password.
- **The agent** (`RemoteSupport.Agent`). A Windows service that runs in the background all the time, even when nobody is logged in. It is what makes **unattended access** possible, it installs and removes the monthly health check, and it **uninstalls itself** if the IT company removes the machine while it is online.
- **The host** (`RemoteSupport.Host`). Started fresh for each session and closed afterwards. It does the real work: takes pictures of the screen, presses keys and moves the mouse, copies files, runs scripts, captures sound, adds the technician's printer. It runs with full system rights so it can also work on Windows' secure screens (the password prompt, Ctrl+Alt+Del, the lock screen). On a free-version computer it also shows the **"Being controlled remotely"** bar for the whole session.

On the technician's computer (one program)

- **The technician console** (`RemoteSupport.Technician`). The list of customer machines, the incoming-call alert, the session window (where the customer's screen appears), chat, tickets.

The server (one program)

- **The platform** (`RemoteSupport.Platform`). A web server that does several jobs at once:
  1. **The public website** — marketing pages, download pages, website chat.
  2. **The portals** — the web pages office staff use for tickets, clients and settings, and the owner portal.

3. **The hub and relay** — every program above connects to it. It rings technicians when a customer asks for help, and during a session it passes the screen picture one way and the mouse and keyboard the other. **It does not store the screen picture.** It just passes it on.
4. **The database** — companies, staff, customers, machines, tickets, photos, chat history. On the live server it is **PostgreSQL** (since 9 October 2026), on a disk of its own; a developer's PC and most test tools use **SQLite**, a database in a single file, and the code works on both.
5. **Payments** — card payments through PayFast, and the owner pages with the control dashboard (V2.2).

There is also an older program, `RemoteSupport.Server`, from before the product served many companies. It is kept but no longer used live. Leave it alone unless you are removing it.

Shared libraries (no program of their own)

- `RemoteSupport.Shared` — the "language" all the programs speak to each other, plus things every program needs (security rules, the two-factor code maths, device identity).
- `RemoteSupport.UI` — the look of the Windows programs (rounded buttons, colours, light and dark).
- `RemoteSupport.Media` — video and sound: turning the screen into compact video (H.264), playing it back, recording sessions, the microphone with echo cancelling.

How a support call works, step by step

1. The customer clicks **Request Support** in the tray app and types what is wrong.
2. The tray app tells the server. The server **rings every technician** at that IT company who is signed in. A large alert appears in the middle of each technician's screen, even if the console is minimised and the sound is off.
3. A ticket is opened automatically with the customer's description.
4. The first technician to click **Accept** gets the call.
5. The server tells the customer's computer to start a **host**. The host connects to the server.
6. The customer sees a **permission prompt** ("Allow a remote support connection?"). If nobody answers in time, the answer is no.
7. Once allowed, the technician sees the customer's screen and can use the mouse and keyboard. Other technicians can join the same session.
8. When the session ends, the ticket is updated with what happened (who connected, chat, files, scripts run, how long it took).

Unattended access is the same, except the technician starts it from the machine list, and the customer is not asked each time — because the customer has ticked *Allow unattended access* in their tray app. If the machine has an unattended password, the technician types it once and it is remembered for them. More in document 02.

On the Free and Basic versions there is no call: the one person picks one of their computers, types its password the first time, and is in — or, on Basic, connects to one without a password and the person there is asked. The computer shows *Being controlled remotely by …* at the top of its screens while they are.

How the programs talk

- Everything travels over **one secure web connection** (HTTPS and secure WebSockets on port 443). Nothing needs a special port opened on the customer's network: every program dials **out** to the server, the way a web browser does.
- There are three kinds of connection on the server:
  - `/ws/device` — each customer computer (the tray app and the agent) keeps one open.
  - `/ws/tech` — each signed-in technician console keeps one open.
  - `/ws/session` — one for each live session, for the host and each technician viewing it.
- Messages are small pieces of JSON text. The screen picture, file contents and sound are sent as compact binary pieces. The definitions are in `src/RemoteSupport.Shared/Protocol.cs`.

How one server serves many IT companies

- Each IT company has a **tenant key**: a long random code built into its downloads. When a customer computer connects, it shows this key, and the server knows which company it belongs to. The key is an **address**, not a password — it is public by design (see document 03).
- Inside the server each company gets its **own separate hub** in memory. Companies cannot see each other because they are not in the same place at all. The hub also knows whether its company is on the free version, and applies those limits itself (`Hub/TenantHub.Free.cs`).
- Every company downloads the **same program files**. Only the branding file (name, colours, logo, server address, tenant key) is different. That is what makes one code-signing certificate cover every reseller.

02 — Every feature, and how it works

Each feature has three parts: **what it does**, **how it works** in plain terms, and **where it lives** in the code (for developers). A mark after the title says how far it has been proven:

- **[Proven]** — used on real computers and confirmed working.
- **[Built]** — written and passed its automatic tests, but **not yet tried on real customer computers**. Treat these as "probably works, needs checking". The checklist is `docs/TEST-3.6.md`.

Document 07 has the full picture of what is proven and what is not. **"V2"** after a mark means it was added between V1 of this pack (2 October 2026) and V2 (6 October 2026) — document 10 lists them all; **"V2.1"** (or "after V2") means it was added on 7 October 2026 — document 11; **"V2.2"** means it was added between 8 and 15 October 2026 — document 12; **"V2.3"** on 16 October 2026 — document 13; **"V2.4"** between 17 and 27 October 2026 — document 14; **"V2.5"** on 28 October 2026 — document 15.

Part A — Remote control

A1. Seeing the customer's screen [Proven]

What: The technician sees the customer's screen live, on any of their monitors, docked in the console, in its own window, or full screen.

How: The host takes a picture of the screen many times a second using Windows' own fast screen copy (*Desktop Duplication*). It compresses the pictures into video using **H.264** — the same compression as online video — using the computer's graphics chip where it can. Only what changed is sent. Scrolling a document went from 2.8 MB per second to under 0.2 MB per second when this was added. If a computer cannot do H.264, it falls back to an older method (small JPEG squares of the parts that changed), so it still works, just using more data.

Automatic quality: if the internet line is struggling, the host lowers the picture quality and frame rate on its own, and raises them again when the line recovers (`LinkController`).

Fixed 6 October (V2): on computers whose graphics chip converts colours for the compression, a program opened or closed sometimes did not appear until the technician pressed Refresh. The converter was asked for its result only once per picture; it is now asked until it has finished (`Media/GpuColourConverter.cs` , `HANDOVER.md` §99). Not yet seen on the affected machine.

A picture that never stays blurry (V2.2, Built): on one test machine the picture arrived blocky and never became clear. Two causes: an automatic "start again" picture every 24 frames threw the sharpened picture away, and nothing sharpened a screen that had stopped changing. Now those pictures are a minute apart, and **a still screen is encoded again for two seconds after any real change** so it comes out sharp (`Host/StillRefiner.cs`). A blinking cursor does not count as a change. When the server has to drop a

picture for a slow technician, it asks the customer's computer for a fresh one. Found on the way: a Windows structure had been read wrongly since September (`HANDOVER.md` §142).

Where: `Host/DxgiCapture.cs` , `Host/ScreenCapture.cs` (fallback), `Media/H264Encoder.cs` , `Media/H264Decoder.cs` , `Technician/ScreenView.cs` .

A2. Mouse and keyboard [Proven]

What: The technician's mouse and keyboard control the customer's computer, including on several monitors, and the customer's real mouse pointer shape is shown.

How: The console sends each mouse movement and key press to the host, which "presses" them on the customer's computer using Windows' input function (`SendInput`). Because the host runs with full system rights, this also works on programs running as administrator, on Task Manager, and on Windows' password prompts (UAC) and the lock screen.

Windows shortcuts (added 2026-10-07, after V2; Built): while the picture has the keyboard focus, the Windows key and its shortcuts, Alt+Tab, Ctrl+Esc, Ctrl+Shift+Esc, Alt+Esc and Print Screen act on the customer's computer, not the technician's. Windows keeps these keys for itself, so the console uses a keyboard hook that exists only while the picture is focused and takes only those keys. Ctrl+C, Ctrl+V and every other key travel exactly as before. There is no on/off switch. Win+L and Ctrl+Alt+Del cannot be taken by any program; the toolbar has buttons for both. Handover §110.

Where: `Host/InputInjector.cs` , `Host/DesktopFollower.cs` , `Host/CursorShape.cs` , `Technician/ScreenView.Shortcuts.cs` .

A3. Several technicians on one session [Proven]

What: More than one technician can watch and help in the same session.

How: The server simply sends the picture to every technician watching and passes every technician's input to the host.

A4. Surviving a dropped connection [Proven]

What: If the internet drops briefly, the session reconnects on its own.

How: The console keeps retrying for two minutes; the server keeps the session waiting for 90 seconds. If the line comes back in that time, the session carries on.

Where: `Technician/SessionLink.cs` , the hub's grace period in `Platform/Hub/TenantHub*.cs` .

A5. System buttons [Proven]

What: From the session toolbar: **Ctrl+Alt+Del**, **Lock the machine**, **Open Task Manager**, **Restart the machine** (and reconnect automatically after it comes back).

How: The host carries these out with system rights. Ctrl+Alt+Del needs a Windows policy setting, which the customer installer switches on.

A6. Copy and paste, both ways [Proven]

What: Text, files and whole folders copied on one computer can be pasted on the other, in either direction, as if both were the same computer.

How: Text is simply sent across. Files are different: when you copy a file, Windows only remembers *where it is*, not its contents. So the product first **sends the file contents across**, then tells the other computer's clipboard where the copy landed. Pasting then works normally in Windows Explorer.

A small helper program running **as the logged-in person** (not as the system) does the final clipboard step on the customer's side, because Windows Explorer refuses to paste a file list put there by a more powerful account. This took several days to discover and is written up in `HANDOVER.md` §39 and §40.

Privacy: the clipboard is only shared while a technician is connected, and stops the moment they leave.

Every movement is written on the ticket (what, which way, how big, when) [Built].

Where: `Host/ClipboardAccess.cs`, `Host/UserProcess.cs`, `Shared/FileSetTransfer.cs`, `Shared/ClipLedger.cs`.

A7. Moving files: drag and drop, and the Files window [Proven, drag-off-the-picture Built]

What:

- Drag a file or folder from the technician's desktop **onto** the customer's screen — it lands on the customer's desktop. [Proven]
- The **Files** window shows the customer's drives and folders; drag files out of it to the technician's computer. [Proven]
- Drag a file straight **off the picture** of the customer's desktop onto the technician's desktop. The product reads what the customer has selected in Explorer. [Built]

How: Files are sent in small pieces (64 KB). Every received path is checked so a file can never be written outside the folder it was meant for (this blocks a known type of attack). The customer is told in plain words what was moved.

Where: `Host/FileTransfer.cs`, `Host/ExplorerSelection.cs`, `Technician/DragOut.cs`.

A8. Running scripts [Proven]

What: The technician can run a PowerShell or batch script on the customer's computer and watch the output appear live.

How: The host saves the script to a temporary file and runs it **with system rights**, sending back each line of output and the final result code.

Where: `Host/ScriptRunner.cs`.

A9. Hearing the customer's computer [Proven]

What: Listen plays the customer's computer sound on the technician's speakers. The **Sound** window explains *why* there is no sound on the customer's machine (muted, wrong speakers chosen, driver problem) instead of leaving the technician guessing.

Where: `Host/AudioCapture.cs`, `Host/AudioDiagnostics.cs`, `Technician/AudioPlayer.cs`.

A10. Talking to the customer, both ways [Built]

What: Talk to the customer sends the technician's voice to the customer's speakers. The customer is asked whether to use their microphone too — **every session, and never for unattended sessions**. Echo cancelling is used at both ends.

Where: `Host/VoiceCall.cs`, `Media/EchoCancelledMic.cs`, `Technician/SessionForm.Voice.cs`.

A11. Privacy screen [Built]

What: Blanks the customer's monitors (they see black) while the technician still sees and works normally. Useful in open offices.

How: Black windows that Windows is told to leave out of screen capture. If it cannot work properly on that computer, it **refuses** rather than half-working, and says so.

Where: `Host/PrivacyScreen.cs`.

A12. Recording a session [Built]

What: Tools → **Record this session**. A red marker shows while recording. The video is saved by the technician's console and uploaded to the session's ticket. Only someone in the session can attach one; only a supervisor or the owner can delete one. The customer is told.

Where: `Media/ScreenRecorder.cs`, `Technician/SessionRecorder.cs`, `Platform/Services/RecordingService.cs`. Recordings are stored in the server's `data/recordings`.

A13. Session chat and the Info window [Proven]

What: Chat with the customer during a session (it pops up on their side and can be moved). **Info** shows the customer computer's details: Windows version, processor, memory, disk, and which account the host is running as.

A14. Remote printing, both ways [Built — V2]

What: Tools → **Print on my printer**: a printer named after the technician (for example "Sue Tech's printer") appears on the customer's computer, and whatever the customer prints to it comes out on the technician's own printer. Tools → **Print a file on their printer...**: the technician picks a file and it prints on the customer's default printer.

How: No printer driver of ours: the added printer is Windows' own *Microsoft Print to PDF* writing to a file, which the host picks up and sends to the console, which prints it. The printer is removed when the technician leaves the session. Known limit, said to the technician: two documents printed within a moment can arrive as one.

Where: `Host/PrintRedirect.cs`, `Technician/SessionForm.Print.cs`, `Media/PdfPages.cs`. `HANDOVER.md` §95.

A15. Wake-on-LAN [Built — V2]

What: Right-click an **offline** machine → **Wake up**. Another of that customer's machines on the same network sends the wake-up signal — nothing to set up on the customer's router. The technician is told which machine sent it, or why it could not be sent.

Where: `Platform/Hub/TenantHub.Wake.cs`, `Shared/WakeOnLan.cs`. `HANDOVER.md` §94.

A16. "Being controlled remotely" [Built — V2, the Free and Basic versions only]

What: On a Free or Basic computer, which has no tray app, a small amber bar at the top centre of every monitor says *Being controlled remotely by* … for the whole session, naming who is connected.

How: Like the privacy screen, the bar is left out of screen capture, so it is on the person's screen but **not** in the technician's picture, and clicks pass straight through it.

Where: `Host/ControlledNotice.cs`. `HANDOVER.md` §104.

A17. Two machines at once, and copying straight between them [Built — V2.2]

What: A technician can have **two sessions open side by side**, one per monitor — two at most (`Shared/SessionLimit.cs`; the console refuses a third before asking, and so does the server; reconnecting to one already open never counts). **Copy on one machine, paste on the other:** files copied on machine A are held, and when the technician clicks into machine B's window and pastes, the server joins the two sessions for that one copy and **A's computer sends the files straight to B's** — through the server, but never through the technician's PC or line. A key made for that copy is handed to each machine inside its own sealed session, so

the server cannot read the files. If an older program is involved, or the server says no, the files go through the technician's PC as before.

Where: `Platform/Hub/TenantHub.Bridge.cs` (`BridgeProblem` is the one rule), `Host/HostEngine.Bridge.cs`, `Shared/BridgeSeal.cs`, `Technician/SessionForm.Bridge.cs`, `SessionForm.Between.cs`. `HANDOVER.md` §143–144.

Part B — Access and permission

B1. Asking for help (attended support) [Proven]

What: The customer clicks **Request Support** in the tray app and must say what is wrong. Every signed-in technician at that IT company is rung. The reason is the biggest text on the alert.

How: The alert is always on top, flashes the taskbar, and never steals the keyboard from whatever the technician is typing. The first to accept gets the call.

Since 9 and 13 October (V2.2, Built): the request has a short **subject** and an optional **description**, and a tick to send **a picture of the screen**, previewed first, which lands on the ticket. The app **remembers the customer's name** (per Windows user). Because customers usually minimise everything to reach the desktop shortcut, the picture often showed only the desktop: the window now **says so**, and **Take a new picture** hides it while they bring the right window up. It never restores their windows for them, on purpose (`Client/ScreenSnap.cs`, `Client/LastContact.cs`, §122 and §129).

Since 18 and 23 October (V2.4, Built): the name typed into a request **names the machine** while it is not linked to a client — shown in grey italic in the console and the portal instead of DESKTOP-ABC123, the latest name winning — and linking it offers that person as a contact (`Shared/OwnerNames.cs`, §150). The **installer asks who the computer is for** (name, surname, email) and that does the same from the start; the server keeps it once per install, so a later request's name still wins (`Shared/GivenPerson.cs`, §156).

Where: `Client/RequestDialog.cs`, `Technician/IncomingCallPopup.cs`.

B2. The permission prompt (consent) [Proven; the system-level version Built]

What: Before anyone sees the customer's screen in an attended session, the customer sees a branded prompt naming the technician. If nobody answers within the time limit (60 seconds by default), the answer is **no**.

How: The host connects to the server first but **does nothing at all** until the customer says yes. The technician sees "Waiting for … to accept" with a countdown and a Cancel button.

Since 16 October (V2.4, Built): the prompt **comes to the front and stays there**. Windows takes the always-on-top mark off a window a background program shows, so it used to open behind whatever the person was

using (`UI/ComeForward.cs` , §149).

Where: `Host/ConsentPrompt.cs` , `Technician/ConsentWaitForm.cs` .

B3. Unattended access — a switch the customer holds [Proven]

What: A technician can connect without anyone at the computer — but **only if the customer has ticked "Allow unattended access" in their tray app**. They can untick it at any time.

How: Unattended access needs **both** the customer's permission (stored on the server) **and** the agent service running and connected. The console shows which machines can actually be reached unattended right now. If the agent is not connected, the server says so instead of silently falling back to asking the customer.

A reinstalled machine waits for the office (V2.1, Built): a machine recognised after a reinstall keeps its history and its tick, but goes attended until a supervisor or the owner confirms it is the same machine — right-click → Confirm in the console, or the Devices page. `HANDOVER.md` §111.

Where: `Agent/AgentService.cs` , `Agent/SessionLauncher.cs` , permission rules in the hub (`TenantHub.MayGoUnattended` is the one question every unattended route asks).

B4. An unattended password [Built]

What: The customer can set a password on their own machine in the tray app. Then nobody — **including the IT company's owner** — can connect unattended without typing it. This is meant for people who connect to their own home PC from the office. There is deliberately **no override** in the office; a forgotten password is removed at the machine itself.

How: The server stores only a scrambled one-way version (a "hash") of the password. Five wrong tries lock that machine for 15 minutes.

Remembered, per technician (V2): once a technician types a machine's password correctly, they are not asked again — on any computer they sign in on. A colleague who never typed it still is. The server keeps only *that it was typed right* (never the password); changing or removing the password at the machine forgets it for everyone, and so does removing the technician. Right-click → **Forget saved password** forgets one's own; a supervisor or the owner can see **who** has it saved (names only) and clear one or all. `HANDOVER.md` §100.

Where: `Client/UnattendedPasswordDialog.cs` , `Shared` (`UnattendedPasswords`), the gate in the hub's connect code, `Platform/Hub/TenantHub.SavedPasswords.cs` .

B5. Exit means offline [Built]

What: If the customer chooses **Exit** in the tray app, the machine goes offline for remote support completely — the agent stops accepting sessions too — until the tray app is started again.

B6. Outside help — let in by the customer with a code [Built]

What: A customer sometimes needs help from somebody OUTSIDE their IT company — their accountant, or another IT company that does something different. Where the IT company has switched it on (off by default; per company and per machine, supervisors and owner only), the person at the computer makes a **9-digit code** in the tray app ("Let someone else connect…"), after a warning about scammers. It lasts 10 minutes and works once. A technician of **another company on the platform** types it in their own console ("Connect with a code…"). The person at the computer is **always asked** to Allow, told by name and company that this is not their IT company. The visitor gets the **screen, mouse and keyboard only** — no files, scripts, clipboard or chat. The IT company's consoles get a pop-up and can **watch** or **end** it; when the visitor leaves, the session ends. Both companies keep a record.

A **guest with no account** (the accounting firm) downloads the **guest console** from `/guest` — nothing to install, no sign-in — and types their name and the code; everything they are shown as says "a guest — not checked".

Or they choose **Connect in this browser** on the same page (`/guest/web`) and do the same with nothing downloaded at all.

Files only if the customer ticks *Also let them copy files* in the Allow prompt, for that session — and then only files, never the clipboard or anything else. (The browser page has no file tool; it says to use the guest console.)

The technician asks, too (22 October, V2.4, Built): in a session, **Tools → Get outside help…** makes a code for that session and a message to Copy, WhatsApp or Email (the browser link carries the code after `#` , so it is in no server log). Whoever types it waits for **the technician who made it** to let them in, files on a tick. **The person at the computer is told, not asked** (Clint's decision). The session stays the IT company's: the outsider leaving or being removed takes only them out, and when nobody of the company is left they go too. Each company can switch this off on Portal → Outside help (on by default); it does not depend on the customer's own outside-help switch (`Hub/TenantHub.Invite.cs` , §155).

Where: `Shared` (`OutsiderAccess`), `Platform/Services/OutsiderCodeService.cs` , `Platform/Hub/TenantHub.Outsiders.cs` (the allow-lists), `Client/OutsiderCodeDialog.cs` , `Technician/CodeEntryDialog.cs` , `Technician/VisitorPopup.cs` , `Technician/GuestForm.cs` , `Platform/Hub/GuestVisitor.cs` , `Platform/wwwroot/guestview.js` . Handover §87, §89, §90, §91 and §92.

B7. Removing a machine, with a reason [Built — V2]

What: For when machines are upgraded, replaced, or were somebody's private device. A **supervisor or the owner** right-clicks a machine in the console (**Remove this computer…**) or presses **Remove** beside it on the portal's Devices page, and picks why: **Upgraded, Replaced, Private device, No longer a customer, or Other** (Other needs a note).

How: The machine leaves every list at once. Its **tickets, sessions and reports stay**. The removal is recorded, and the Devices page ends with a **Removed machines** list: what, when, who, why and the note. If the machine is **online**, it **uninstalls itself** (the service, the files, the shortcuts and its health check). If it is **offline**, it

is refused when it next connects, and the program stays on it until somebody removes it. A removed install can never put itself back; installing again adds it as new.

Where: `Platform/Hub/TenantHub.Free.cs` (`RemoveDeviceAsync`, one removal for both versions), `Shared` (`RemovalReasons`, `ClientUninstall`), `Agent/AgentService.cs` (`UninstallSelf`), `Technician/RemoveDeviceDialog.cs`, `Pages/Portal/RemoveDevice.cshtml`. `HANDOVER.md` §106.

B8. Uninstall, from the tray app [Built — V2]

What: A customer who no longer wants the product chooses **Uninstall…** in the tray menu. It asks once; Windows then asks for administrator permission (No changes nothing). The program is removed, and the IT company sees the machine leave its list as **"Uninstalled at the machine"**, by the person who was signed in there.

Where: `Client/MainForm.cs` (`UninstallAsync`), `deploy-client/Uninstall-Client.ps1` — the same script the agent uses, now kept in the program's folder for this. An install made before 6 October does not have it, and says so. `HANDOVER.md` §106.

Part C — The customer's side

C1. One download per IT company [Built]

What: Each IT company has a page, `theremotesupportprogram.com/get/<company>`, with its own logo and colours. The customer downloads **one .exe file**, which offers **Get help now** (run without installing) or **Install**.

How: Every company's .exe is the same file. When it runs, it works out which company it came from (from the download's own name and Windows' download record) and fetches that company's branding. This is what lets one signature cover every reseller.

Where: `Client/Launcher.cs`, `Client/LauncherForm.cs`, `Shared/DownloadIdentity.cs`, `tools/Pack-ClientExe.ps1`, `deploy-client/Install-Client.ps1`.

And afterwards (7 October 2026, HANDOVER §113, not yet seen on a real machine): when a company changes its logo, colours, wording or permission-prompt settings, every installed machine takes the new look — at once if it is on, otherwise when it next connects. The server sends only what a person sees, never where the machine connects or whose it is. The machine keeps it beside what it was installed with; the tray app restarts itself quietly to show it, only when nobody is using it. `Shared/BrandRefresh.cs`, `Platform/Hub/TenantHub.Branding.cs`, `tools/brandpush`.

C2. The tray app [Proven; Uninstall… Built in V2]

The menu: **Request Support**, **Chat with a Technician**, **Allow unattended access**, **Unattended password…**, **Let someone else connect…** (only where outside help is on), **Uninstall…** and **Exit**. Double-click opens the

window. A small strip in the corner shows while a session is live.

On the **Free and Basic versions** there is no tray app at all: the downloaded file asks for the computer's password (on Basic it may be left blank — then the person there is asked each time, by the background service), installs only the background service, and leaves nothing on the screen (Part H).

C3. Support in a web browser [Built]

What: `…/help/<company>` — a customer without the app can fill in a form, ring the technicians, and share their screen from the browser. **View only**: no browser lets a website control a computer. They become a contact and a ticket. Off by default; switched on with opening hours in the portal.

Where: `Platform/Hub/TenantHub.Web.cs`, `wwwroot/help.js`, `Pages/Help.cshtml`.

C4. Website chat [Built — needs the live site]

What: An IT company pastes one line of code onto its own website. A chat bubble appears; visitors give a name and an email or phone number and chat. It rings everyone signed in, and **the first to reply takes it**. If nobody is available or it is outside hours, the message becomes a ticket. Since 2026-10-15 a chat can be **handed over** to anybody else signed in — by the person answering, a supervisor or the owner, or by anybody while nobody has answered; the visitor is told who has taken over, and the new person hears the dings from then on (HANDOVER §145).

Where: `Platform/Services/WebChatService.cs`, `Pages/WebChatFrame.cshtml`, `Technician/WebChatForm.cs`.

C4b. Tickets by email and a contact form [Built — not yet against a real mailbox]

What: A customer emails the IT company's support address, or fills in a contact form on its website, and a ticket opens by itself. If their email address is on file, it is linked to their client. If only their company is recognisable (by the part after the @), the ticket says so and offers one button to add them as a contact. They get an automatic reply with the ticket number, and when they answer that reply it goes onto the same ticket. Out-of-office replies, bounces, newsletters and spam are left alone. Works with any mail host: the platform either reads the company's mailbox (cPanel, Gmail with an app password, Zoho…) without changing anything in it, or the company forwards its mail to a mailbox of ours (the way for Microsoft 365). Microsoft 365 sign-in is planned for later.

Where: `Platform/Services/TicketIntake.cs` (every rule), `Services/MailWire.cs` (reading and sending mail), `Pages/Portal/MailIntake.cshtml`, `Pages/ContactForm.cshtml`.

Updating the client from a ticket (added 9 October 2026, not yet against a real mailbox): under any comment box, *Also email this to the client* (unticked — most comments are internal), and *Email to the client…* on a comment already posted, in the portal and the console. The client gets the fault the ticket was opened for and that ONE comment, with the name of whoever wrote it, from the support address above — never the

rest of the ticket. Their reply comes back onto the same ticket, and everybody on it is flagged until they open it. Offered only once Tickets by email is set up; signup now asks for the address. Every rule is `Platform/Services/ClientUpdates.cs`; section 123 of the handover is the record.

C4c. A knowledge base [Built — not yet on hardware]

What: How things are done, written down once. A **General** section and a **Clients** section (each article is General or belongs to one client). Everybody at the company can read, write and edit; every save is kept and an older version can be put back; only a supervisor or the owner can delete. When a ticket's work is done, the people on it see **Add to knowledge base**, which opens the ticket as a draft article — what was needed and what was done, as numbered steps — to be reworded and saved. Both sections can be sorted into **folders and subfolders** (up to six deep), opened like folders on a PC with the path shown above; a search looks through everything below where you are. Removing a folder never deletes anything — what was in it moves up a level. A long web address can be shown as a **button** with words of your choosing (or a web address on a line of its own becomes an *Open site* button). Screenshots can be pasted straight in. A ticket, a client's page and the technician's session window all say when that client has articles. A line that looks like a password is warned about. Staff only — customers never see it.

Where: `Platform/Services/KnowledgeBaseService.cs` (every rule), `Services/KbText.cs` (how the words are drawn — everything typed is made safe first), `Pages/Portal/Knowledge*.cshtml`.

C5. The monthly health check [Proven in parts]

What: Once a month, a customer machine runs maintenance (with a restore point first) and a read-only health check — disks, drive health, backups, updates, security — and a branded PDF report is emailed to the IT company.

How: It is **off by default**. The IT company ticks *Monthly health check* on a customer's page (usually for service-contract clients); every machine of that customer follows, and a single machine can be switched on or off by itself. The agent service installs or removes it on the machine and reports what it actually found. If the install fails (for example antivirus blocks it for a moment), it retries after 1, 5 and 15 minutes.

The machine stores no email settings — it uploads its results, and the server builds the PDF and sends the mail.

Where: `healthcheck/` (the PowerShell toolkit, with its own README), `Agent/HealthCheckKeeper.cs`, `Platform/Services/HealthCheckService.cs`, `ReportPdfService.cs`.

C6. Keeping itself up to date [Built — V2.3]

What: Every program on a customer's computer — the tray app, the background service, the program that does the work in a session, the install scripts and the health check — updates itself when we release a new version, and so does the technician's console. Nobody at the customer's computer is asked (Clint's decision).

How: Every build carries a version made from the moment it was built (**2026.10.16.1432** is 14:32 UTC on 16 October 2026), shown in the console, the portal's Devices page and the control dashboard. A new release goes to **our own company's** computers first; a Full admin or Owner moves it on to **the pilots**, then **everybody**, on **Owner** → **Updates**, and can pause it. The server offers it to a few computers a minute. Each computer:

1. downloads it from its IT company's own server;
2. checks every file is **signed by us**, not revoked, and **newer** than what it has — anything else is refused, and that version is never taken again;
3. waits until **nobody has been connected for 10 minutes**;
4. swaps the files (a running program is renamed aside, never overwritten; only the release's own files are touched — never the computer's identity, branding or settings) and restarts its service;
5. keeps the new version only if it **connects back within 5 minutes**; otherwise it puts the old files back by itself. A computer that rolls back **pauses the release** for everybody and emails the admins.

The console checks before its sign-in window and restarts itself in the new version; while open, it shows **Update ready**.

Computers installed before 16 October need one update by hand — connect, then **Tools** → **Scripts** → **Update this machine** — after which they update themselves. The portal's Devices page counts and marks them.

Seen on hardware (19 October, V2.4): the background services updated themselves — one PC took four releases, about 20 seconds each, with no rollback. **The tray app never followed**: Windows answered "what version is this file?" from the copy already running, so the tray app read the new file as its own old one. It now reads the file itself (`UpdateTrust.FileVersionOnDisk` — never ask Windows for the version of a file a program started from). Tray apps already running in the field are the old code and cannot notice: **each computer needs one sign-out or restart** after that deploy, then follows by itself. `tools/updatestate.ps1` , run on a computer through Scripts, says why it is behind (§152).

Where: `Shared/AppVersion.cs` , `UpdateTrust.cs` (the signature check), `UpdateRunner.cs` , `UpdateSwap.cs` , `UpdateByHand.cs` ; `Agent/AgentUpdate.cs` ; `Technician/ConsoleUpdate.cs` ; `Platform/Services/UpdateService.cs` (every rule of the rollout), `UpdateSweep.cs` , `Pages/Owner/Updates.cshtml` ; `tools\BuildVersion.ps1` . HANDOVER §147–§148.

Part D — The technician console

Feature	What it does	Status
Machine list	Customer machines in **Attended** and **Unattended** tabs, grouped by customer and the person who owns each machine; online status; processor, memory, disk (red over 90% full), Windows version	Built

Feature	What it does	Status
Incoming-call alert	Centre-screen, always on top, flashes, never steals typing	Proven
Ding and ring	A soft two-note **ding** for every chat message from somebody else (customer, office, website visitor); a soft **ring every four seconds** while a support request or a website visitor waits, stopping the moment it is answered, cancelled or dismissed. Our own sounds, made in code (`Shared/Chimes.cs`), the same in the console, the customer's tray app and the portal. A bell in the header mutes them on that PC (the portal has its own button, per browser)	Built (V2.1)
Tools menu	Chat, Files, Scripts, Sound, Info, Listen, Record, Privacy, Talk, Print on my printer, Print a file on their printer — each in its own movable window that remembers where it was put	Proven / Built
Wake up	Right-click an offline machine (see A15)	Built (V2)
Remove a machine	Right-click → Remove this computer…, with a reason — supervisors and the owner (see B7)	Built (V2)
Saved passwords	Right-click → Forget saved password; supervisors and the owner also see who has a machine's password saved (see B4)	Built (V2)
One look	Every button, box, list and menu rounded, in one colour rule, light and dark; checked window by window by `tools/lookprobe`	Built (V2)
The company's logo everywhere	Every console window — sign-in, sessions, tickets, chat, tools — and the console's shortcuts carry the company's logo, not Windows' default icon (`HANDOVER.md` §114)	Built (V2.1)
Encrypted badge	Each session window says *End-to-end encrypted* (green) or *Not end-to-end encrypted* (an older program is involved); if a customer computer's identity has changed, the console stops and asks first (document 03)	Built (V2.1)
Tickets window	Work tickets without opening a browser; draws exactly what the server says can happen next. The queue's headings sort, as in the portal (§165). New ticket and Edit offer only the categories the company uses, and ask for photos only if it does (Job types, F2b)	Built
Latest activity	Beside Tickets: every update on the tickets as it is posted — comments, stage changes, customer replies, assignments ("assigned it to you"), new	Built (2026-

Feature	What it does	Status
	tickets — newest first; all tickets or only mine; by client and by who posted it; the count of what is new since I last looked moves by itself. A click opens the ticket. A technician sees only what their Tickets list shows (`HANDOVER.md` §164)	10-26)
Link a machine	Right-click a machine to say which customer and person it belongs to, or **Unlink** it (V2.2)	Built
Two sessions at once	Two session windows side by side; copy on one, paste on the other (see A17)	Built (V2.2)
Off means offline	A machine switched off at the wall, asleep or off its network shows offline within about 45 seconds (the server notices the silence, §122)	Built (V2.2)
Reconnecting asks nothing	Connecting again within 30 minutes of the end of the last session carries on the same ticket without asking for a reason (§122)	Built (V2.2)
Chat from the list	Right-click a machine → Chat: the chat box switches to that machine, ready to type	Built
Processes and Performance	Right-click a machine → a live process list (CPU, memory, user) and graphs (processor, memory, disk, network) **without connecting** — nothing shows on the customer's screen, nobody is asked, no password, look only, nothing logged (the owner's decision). The Network card says whether each network is **Private or Public** (Public in amber) and the machine's **IP address**. Figures come from the agent, or the tray app where there is no agent	Built
Office chat	Chat with colleagues one to one or with everyone; @mentions pop up	Built
Light or dark	Each person chooses; applied when the console next starts	Built
Sign out	Header and tray menu. The X button hides to the tray on purpose	Built
Closes to tray	Keeps ringing while hidden	Proven
Download as one .exe	`…/tech/<company>` — sign in, download, it installs itself for that Windows user with shortcuts	Built

---

Part E — The office portal (website for staff)

Signed-in staff use `https://theremotesupportprogram.com/portal`. It also works on a phone: below a certain width, lists turn into cards and the menu folds away.

Page	What it does
Dashboard	Open jobs by status, unassigned jobs, today's call-outs, workshop jobs, stale jobs, machines online, machines not yet linked to a customer
Tickets	All jobs, filters (mine, unassigned, open, by client, by machine) and search. **Every column heading sorts** — click it, click again to turn it round (#, Type, Subject, Client, Assigned to, Status, Priority, Last updated, Due); **Group by customer** is a tick; an orange *Customer replied* while the customer's email is the newest entry (2026-10-26, `HANDOVER.md` §165)
New ticket	Category first, as a row of coloured buttons (each job type its own colour, V2.2), then the client (found by typing), the people assigned, an optional **due date** (§165), and the category's questions
A ticket	A **story**: the stage at the top, a numbered timeline below, a comment box always there (it grows as you type, up to 15 lines), and only the next allowed steps shown as buttons. ✎ **Edit** changes the subject, description, appointment, **due date** (§165) or category; **Back to …** undoes the last step; files and photos can go on any comment (V2.2, Part F)
Latest activity	The same list as the console's (Part D), a tab beside Tickets for every role and a card on the Dashboard, with a count of what is new; each line opens the ticket at that entry (§164)
Clients	Every customer, searchable as you type. Each has a page: contacts (people), machines, open work, history, health check tick
Remove a client	A misspelling or a duplicate: everything on it (tickets, machines, people, knowledge base, recurring tasks) moves to the right client, which keeps its own details and only has its blanks filled; then it is deleted. Owner, supervisor or office administrator
Import clients	From another program's spreadsheet (.xlsx or .csv), with a preview; never overwrites; repairs common errors (lost leading zeros in South African phone numbers, garbled letters)
Devices	All machines, grouped by customer, unlinked ones last. Supervisors and the owner can **Remove** a machine with a reason; the page ends with **Removed machines** (V2)
Items and Suppliers	A shared list of parts, which refuses near-duplicates ("DDR memory" beside "memory DDR"); suppliers with their email addresses

Page	What it does
Workload	Who is carrying which jobs, and the jobs nobody has
Sessions	Past remote sessions
Health checks	Every report received, its findings, the PDF, whether the email went
People	Add staff by name and email (they are emailed an invitation), tick their roles, reset their two-factor, sign them out everywhere
Branding	Logo, colours, support email and phone, permission-prompt settings, with a live preview
Job types	Which categories and which steps the company uses, card by card, with the way a job goes under each; a supervisor or the owner saves it (see F2b)
Downloads	The two links to hand out: customers' and technicians'
Browser support, Website chat	Switch on, set hours
My account	Change your own password (needs the current one and a code)
Billing	Plan, status, how to pay, the terms accepted and every price agreed; since V2.2 the card on file (PayFast), payments received, and cancelling (G6)
Write a review	A week after joining, the portal asks once for a short review (G8)

The menu is a row of **tabs** across the top (since 9 October), the current page filled in.

A **free-version** company has a much smaller portal: Your computers, Downloads, My account and Upgrade (Part H). Every other page sends it back to Your computers.

Part F — Tickets and job routes

F1. Tickets open and fill themselves in [Built]

Every remote session gets its own ticket automatically. Accepting a call assigns it. Connecting, chat, scripts, clipboard and file movements, and the session's end and length are all written on it. Bigger jobs can be made of several sessions (a "parent" ticket). Staff can also open tickets by hand.

Since 9 October (V2.2): **linking a ticket to a bigger job no longer closes it** — it stays open, is finished and closed on its own, and the bigger job **cannot close until its parts are done** (a pop-up names them; `TicketService.ClosesJob`, `Shared/JobParts.cs`). A ticket can be marked **Part of / Not part of** the job, and unlinked.

F2. Six job categories, each with its own route [Built]

A **route** is the set of stages a job moves through. Each stage says what must be done before the job can leave it, and who it lands with next. The routes were written by Clint and are recorded in `docs/NEXT-SESSION-WORKFLOWS.md` in his words. All the rules live in **one file**, `Platform/Services/Workflow.cs`, and both the portal and the console just draw what it allows.

Category	The route, simplified
General (the default)	Done (closes) / Quote needed / To be invoiced. No client needed.
Remote support	To be invoiced / Do not invoice (service-contract client) / Do not invoice (needs supervisor approval) / Link to another open ticket of the same client
Walk in	Repaired at counter → to be invoiced / paid at counter / no invoice — or booked into the workshop
Workshop	Booked in (with at least three photos of the machine as it arrived) → Inspection findings → Repairable / Not repairable → quote, parts, fitting → Solved, to be invoiced (with what was done, time and amount) → invoiced → paid → collected (three photos out) or delivered. **Not repairable** offers an inspection fee, do not invoice, or (V2.2) **Damage report needed**: a note for the client, the report attached (required), time and amount, emailed to the client by default — never the time or the amount — then to be invoiced
Call out	Booked (can be rescheduled: new date and time, and why) → Work done → to be invoiced / paid on site (cash or card) — or the same ticket becomes a Workshop job
Recurring task (added 2026-10-07, after V2)	Set up ONCE — daily (or weekdays), weekly on chosen days, or monthly on a date or the last day — for maintenance, billing, readings or a type the company adds. A ticket opens for its people at 06:00 company time on each day it is due → Done (closes) / To be invoiced; the office may press both. The next opens even if the last is open (it shows as Overdue). Cancel stops future tickets only. Rules in `Platform/Services/RecurringTaskService.cs`.

Common pieces across routes:

- **Quotes:** request prices from suppliers by email (one supplier in To, or several in Bcc), with the machine's details, and asking the delivery fee, whether each item is in stock, and the ETA. Each item can carry several photos of the part (up to 12), all attached to the email. The email is signed with one of the company's **signatures** (Suppliers page: lines of text, each with its own font, size and colour, and/or an image); with more than one, the person is asked which, and the one they used last is already ticked. Quote → sent → accepted (invoice, wait for payment, order parts, parts received, fitted) or rejected.
- **Billing is a lane, not a category:** office staff handle "To be invoiced", "Invoiced — awaiting payment" and closing. **A technician never closes a ticket.** When a job lands in billing, every office administrator (or, if there are none, the supervisors and owner) gets a pop-up, and a badge counts the jobs waiting for the office until they are dealt with.
- **"Do not invoice"** needs a supervisor's approval (except for service-contract clients).
- **Delivery instead of collection**, with the delivery's time added to the job.
- **Several machines:** booked in together as separate linked jobs, or as **one job** (for example a data transfer from an old PC to a new one — three photos of each).
- **Several people on one ticket**, all equal. Anyone working on the machine is added.
- **@mentions** stay flagged until the person replies; the console's Tickets button shows a count.
- **Photos** are shrunk before upload (2048 pixels, JPEG), uploaded one at a time with progress.
- **Workshop device passwords** are stored **encrypted** and shown only behind a Show button.
- **Abandoned** is available from any stage, with a reason.
- **Go back a step (V2.2):** every stage has **Back to …**, with a reason. Every stage move writes a snapshot of the job before it (`TicketNote.Before`), and Back puts every route field back; the undone step stays in the story, crossed through. It warns (never refuses) when an invoice, payment or email went with that step. A closed job is reopened only by a supervisor or the owner. **Any new code that moves a stage must write the snapshot** (`Services/TicketService.Back.cs` , §128).
- **Edit a ticket (V2.2):** subject, description, appointment and category, by anybody who can see the ticket; each save is one story entry saying what each field was. A new category restarts that route at its first stage; making it a Workshop job needs a technician and three photos in (`Services/TicketService.Edit.cs` , §130).
- **Files and photos on any comment (V2.2):** each uploaded as it is picked (photos made smaller first), 20 MB each, a comment may be files alone, and they go as attachments when the comment is emailed to the client (15 MB together at most, refused before anything is posted). Pictures and PDFs open in the browser; anything else downloads (`Services/TicketAttachmentService.cs` , §128).
- **Emailing the client from a ticket** — see C4b.

F2b. Job types: each company chooses its categories and steps [Built 27 October, not on hardware]

Clint: some IT companies "*wont want all the categories, or in some instances may not want all the stages of a category*" — for example no before-and-after photos — and they must be able to say so "*without breaking the flow.*"

- **Portal → Job types.** A card for each category, its steps in order: a tick box for each one that may be switched off (saying what happens with it off), and a padlock, greyed, with the reason, for each one that may not. Under each card, **the way a job usually goes**, redrawn as boxes are ticked — the server works it out, so it always matches what saving will do. Everybody may read it; **a supervisor or the owner** saves it.
- **What can go (24 switches):** the categories Workshop, Call out, Walk in and Recurring task (**Remote support and General never **— sessions, emails and website chats open them by themselves);** photos when booked in **and** photos when collected or delivered**; skippable steps (inspection findings, asking suppliers for prices, payment before ordering parts, tracking payment after invoicing, supervisor approval for *Do not invoice*); and single choices (damage report, inspection fee, waiting on customer, out for delivery, another machine, quotes, reschedule, paid on site, paid at the counter, needs workshop work, needs to be booked in, the SLA button, link to parent).
- **Nothing can break a route.** Before anything is saved one check walks every route and refuses a set of switches that would leave a job with no way forward, or let one close without ever being invoiced or deliberately not. The locks exist so no combination is refused; a test walks every combination (25,652 routes).
- **A job finishes the route it started on.** Each ticket keeps a copy of the switches from the day it was opened; only new tickets follow a change. A category switched off takes no new tickets, but its open ones finish.
- **Shown once to the owner** the first time they open the portal after it arrived, with *Keep everything*. **Once means once** (V2.5, 28 October): it used to send them back on every Dashboard click until they saved, which looked like the Dashboard tab opening Job types; now the trip itself is remembered. Not on the live server until the next deploy.
- New ticket, the ticket page, Edit and **the console** follow it at once — an open console is told the moment the page is saved.
- **The rules are in one file**, `Platform/Services/JobTypes.cs` (`HANDOVER.md` §166–§168; the brief with Clint's answers is `docs/NEXT-SESSION-JOBTYPES.md`).

Part G — Running the business

G1. Many IT companies on one platform [Proven]

Each company has its own branding, people, customers, machines and tickets, and cannot see anyone else's. See document 01.

G2. The owner portal [Built]

`…/owner` — sign in with the owner key **and** a code from an authenticator app, or (since 13 October) as a named **platform admin** with your own email, password and code. Approve companies, see all companies, change seats, status (Trial, Active, Suspended) and country, and set the platform's own email account (used for trial notices, invitations, new-sign-in alerts).

Since V2.2: Admins (four levels: Viewer, Manager, Full, Owner; set up from a mailed link; a log of everything by name — `Services/PlatformAdmins.cs`); **the control dashboard** — a card per server with its specifications and five readings, amber at 70% and red at 85% of what a server was measured to carry, emailed to the admins when it crosses a level, and every company's people, technicians and machines; a company on another server is changed through that server's own rules (`Services/ControlServers.cs`, §133). **Pilots** (G7), **Reviews** waiting for approval (G8), **PayFast** (the account and a sandbox test checkout), and **Preview** (see the hidden real website). **Suspending** a company now takes effect at once: its consoles and machines are turned away and its people signed out, and the owners are emailed.

G3. Signups, trials and notices [Built]

Public signup is **switched off** while the product is tested privately — **until 1 November 2026**, when the deploy with `-RealSite` opens it together with the real website (since 27 October the two cannot disagree: every deploy writes both, so **every deploy after 1 November needs** `-RealSite`, §169). When on, a company signs up in steps (added 2026-10-05): **where** it is (South Africa, United States, Canada, United Kingdom or Europe — which sets the price, the currency and the server its data lives on), **Free or Full** (added 6 October, V2), **the terms** for that region (privacy law, "we are not responsible for loss of data", and so on, in `Platform/Terms/`; the free version has its own short section ahead of them). Since 6 October **no terms are marked DRAFT** — Clint: "*assume they are correct as is*"; his lawyer's changes, when they come, are edits and a new version number. Then **the company, its time zone and its owner's login** (the US, Canada and Europe choose a time zone; every time on the portal is shown in it, never the server's). The exact terms they accepted are kept on their account. A region with no server yet takes their details for a **waiting list** instead. Then it waits for approval, gets a 14-day trial, and is emailed before the trial runs out. While signup is off, new companies are created on the server's command line (`create-tenant`, then `invite`).

G4. Roles as tick boxes [Built]

A person can be any mix of **Owner**, **Supervisor**, **Technician** and **Office administrator**. Every role uses a paid seat (since 2026-10-05); one owner with no other box is free. A company may have several owners (since 2026-10-08): only an owner makes, changes or removes one, every owner-only login past the first takes a seat, and there is always at least one. Office administrators can never connect to a customer's machine. All permission questions are answered in one place, `Platform/Services/Permissions.cs`.

The free owner-only login oversees (since 15 October, V2.2): it runs the company (billing, people, branding, clients and imports, items, suppliers, the knowledge base), sees every ticket and may **comment and @mention** — but does **not** work tickets: no moving stages, editing, assigning, invoicing, closing, emailing the client, answering website chats, or connecting. Ticking any other box gives all of that back and takes a seat. Enforced twice: `Permissions.CanWorkTickets` in the services, and an **allow-list** on the ticket pages (`Pages/Portal/TicketWorkAttribute`) that refuses any action not named in it (§146).

G5. People choose their own passwords [Built]

Adding a person sends them an **invitation email** with a one-use link (good for 72 hours). They choose a password, set up the authenticator app, and receive ten recovery codes. **Forgot password** works the same way. Nobody ever types a password for somebody else.

G6. Pricing, seats and paying [Built 2026-10-05; FIVE SEATS, MORE ON REQUEST since 2026-10-25; PayFast built 13–14 October]

Five seats; more on request for R500 / $49 / £49 / €49 a month (South Africa / US and Canada / UK / Europe), any mix of roles; one owner-only login is free on top, and everybody else (a second owner-only login too) takes a seat. Every company starts with five (Clint: the product is for small IT companies — "*our vision is to make proper I.T support more affordable for smaller I.T companies*"). **An owner who needs more asks on Billing; a platform admin allows up to a number, or declines, on Owner → Approvals** (or sets it on All companies). Then each seat past five is **R300 / $20 / £20 / €20 a month**, agreed by the owner on the People page before the person is added, and PayFast's amount follows from the next payment; a pilot's extras are invoiced. Without an allowance a sixth person is refused to everybody. **Anywhere IT (marked "not billed") is never limited.** (5–22 October: extras up to ten for anybody; 22–25 October: none at all.) See `docs/PRICING.md` and HANDOVER §160; the prices live in `Platform/Services/Pricing.cs`.

Five companies at a time (2026-10-22): each server lets in five paying companies (not Anywhere IT, pilots or the free version); past that, signup offers a waiting list, and a platform admin offers the next company a place from Owner → Approvals — a link that works once and holds the place for 7 days. The number is set on the same page (`Services/OpenPlaces.cs`).

Paying by card, through PayFast (V2.2, built and proved on PayFast's sandbox; hidden on live): a South African company choosing the full version at signup ends at **PayFast's own page**: R0 now, the package from 14 days on, the card held by PayFast. PayFast's notice switches the company on and emails the owner — no approval by hand. The owner can **add or change the card** on Billing, and Upgrade (from the free version) goes on to PayFast. **The amount is kept true**: when people are added or removed, PayFast is told the new amount for the next charge (an hourly sweep retries). Our own receipt goes out for each payment, and Billing lists them. **A payment that does not come** (PayFast sends no notice for a failure, so we ask PayFast's API): reminders at 1, 7 and 13 days with PayFast's change-the-card link, **suspended when more than 14 days late**, lifted by a payment. **Cancel** on Billing tells PayFast first; service ends the day before the charge it was waiting for. Every rule: `Services/PayfastService.cs` , `PayfastWire.cs` , `CompanyBilling.cs` and `CompanyBilling.Late.cs` (§134–137). Free signups and pilot companies do not use a card.

The **Basic version** (the free version until 2026-10-25) is R50 / $5 / £5 / €5 a month by card. Upgrading from it is the same kind of yes: the owner accepts the price from R50 to the package, and it is recorded on Billing (Part H).

G7. Pilot companies [Built — V2.2, not on hardware]

Clint's deal with the first two IT companies: **five seats free while they are pilots** — no card (PayFast cannot hold a subscription of R0), and since 2026-10-22 nothing to invoice either, as five is the most any company

has. **A pilot is ended by notice** at Owner → Pilots (the terms give 30 days): on the day it is either suspended and closed 30 days later, or moved to R500 with a 14-day trial to add a card; the notice can be withdrawn until then. The page shows each pilot's people, machines, tickets, sessions and last sign-in (§153). A platform admin makes a **one-use link** at Owner → Pilots (or `pilot-link "<company>" [email]` on the server): it works for 30 days, only its fingerprint is kept, and it is shown once. It opens `/pilot/{token}`, which works with signups off and behind the holding page: the pilot terms (`Terms/pilot.txt` ahead of the region's), then the company form. The company is **Active at once — no trial, no approval, no card** — and the link is spent in the same save, so two people cannot both use it. The person signing up is Owner and Supervisor (one of the five seats). All companies shows a pilot's price as "*Pilot · free*", with the end date once notice is given. An existing company becomes a pilot with `set-brand <slug> pilot true`. Every rule: `Services/PilotService.cs` (§146). The guide sent with the link is `docs/pilot-brochure/The Remote Support Program - Pilot Guide.pdf`.

G8. Reviews from companies [Built — V2.2]

A week after joining, the portal asks the company once for a review (*Later* asks again in a month, then *Don't ask again*): stars, a few words, a name and the **company's name**, with a tick to allow publishing. A platform admin approves each at Owner → Reviews before it appears; the real website's home page shows the six newest, safely encoded (`Services/ReviewService.cs`, §139).

G9. The real public website [Built — V2.2, hidden]

About a dozen public pages (the home page, a page for each part of the product, pricing, about, contact, help and questions, terms and privacy), in light and dark, with screenshots made only from the real windows fed made-up companies (`tools/siteshots` — never a real customer's screen), checked for broken HTML, search-engine basics, links, size and stale claims by `tools/sitecheck`. **Hidden behind the holding page until Clint says so** (`Publish-Platform.ps1 -RealSite`); a platform admin can preview it on the live server. The words have not yet been checked by Clint (`docs/WEBSITE.md`). Pages: `Services/SitePages.cs`.

Since 25–27 October (V2.4): it speaks to more than IT companies — *businesses that look after other people's computers and jobs*: IT companies, software and point-of-sale suppliers, in-house IT teams, computer and cellphone repair shops, call-out businesses, and remote workers (with a page of their own, `/remote-workers`) — in the words South Africa searches (*job cards*, *remote access*, *live chat*, from Google Trends, `docs/WEBSITE.md`), and carries **the logo** on every page (`wwwroot/img`, §171). Our own site gets **a chat bubble saying The Remote Support Program**, answered by Anywhere IT's people as their own website chat (`Services/SiteChat.cs`; only for the company named by `Platform:SiteChatSlug`, §169). **Any new public page must be in `go-live.sh`'s holding list** until 1 November, or it is public the moment it deploys (`tools/sitecheck` fails otherwise).

G10. A company takes its data with it [Built — V2.4, 22 October]

What: Portal → **Export** (owner and supervisors) makes one zip, in the background, kept 7 days: the clients as a spreadsheet whose first sheet goes straight back into Import clients, every ticket with its whole story, quote

lines, photos and files, the knowledge base as web pages and text, and — only when ticked — the session recordings. **Never a device password.** A **suspended or closed** company is exported **by us** (Owner → All companies → *Export their data*): its owners are emailed a link to a page with a download button (mail filters open links, so the link alone downloads nothing).

Where: `Services/CompanyExport.cs`, `Pages/Portal/Export.cshtml`. HANDOVER §154.

Part H — The Basic Version for Remote Workers (the free version until 2026-10-25) [Built — V2; R50 since 2026-10-25; none of it on hardware]

Changed 2026-10-25 (HANDOVER §161): Clint ended the free version. It is now *The Basic Version for Remote Workers*: **R50 / $5 / £5 / €5 a month** by card through PayFast after a 14-day trial, **up to three computers**, every other rule below unchanged. The free accounts from before are given 30 days' notice from Owner → Basic version. It never counts against the five places. In the code it is still `FreePlan` and the plan `"free"`; read "two computers" and "free" below as three and R50.

For **people with remote offices**: one person reaching **their own** computers. Clint's fourteen decisions are in `docs/NEXT-SESSION-FREE.md`, and `HANDOVER.md` §100–§107 is the full record.

H1. What it is

- **One person**, who is the owner and the technician. Nobody else can be added.
- **At most two computers**, counted by the actual machine, so reinstalling one is not a third. A computer can be **removed** to free its place, **twice a calendar month** (so two places cannot be passed round a whole customer base).
- **Unattended only.** Nobody at a free computer is ever asked anything, and it cannot ask for help. Every free computer **must have an unattended password**, chosen when it is installed. The technician types it once in the console, and it is remembered.
- **What a session can do:** see and control the screen; Ctrl+Alt+Del, lock, Task Manager and restart; Files, copy and paste, dragging files; printing both ways. **Not:** scripts, sound, talking, recording, the privacy screen, chat, tickets, browser support, website chat, outside help.
- **For their own computers only** — written into its terms. Supporting customers needs the full version. Free for business use, no time limit, no nagging.
- **The server enforces all of it.** The hub refuses a third computer when it says hello, refuses anything the free version does not have whatever the console sends, and a single rule in front of the portal refuses every page that is not the free version's (`Hub/TenantHub.Free.cs`, `Services/FreePortal.cs`, `Shared/FreePlan.cs`).

H2. Signing up

The same signup page: country, then **Free**, then the terms. **No logo or colours** are asked — it uses the product's own colours, light and dark. The company name and phone number are optional. Like any signup it waits for the platform owner's approval, and is then **active, with no trial** (there is nothing to run out).

H3. Adding a computer

On the computer to be added, the owner opens the link from their portal (`/download/<company>`) and runs it. One window, **Add this computer**: the password twice, then:

1. It asks the server whether this computer may be added — a third is told why **before anything is installed**.
2. Windows asks for administrator permission, and **only the background service** is installed — no tray icon, no shortcuts, nothing at logon.
3. It connects once, as the computer itself, to switch unattended access on and set the password.

Changing a computer's password is running the same download on it again.

H4. While connected

The amber **Being controlled remotely by …** bar (A16) is on the computer's screens for the whole session — the only sign, since there is no tray app. It is not in the technician's picture.

H5. The portal and the console

- **Portal:** *Your computers* (each one, online or not, *1 of 2*, removals left this month, Remove, and how to add one), *Downloads* (the console and the add-a-computer link), *My account*, *Upgrade*.
- **Console:** only *Your computers*, Connect (asks the password only the first time — no reason, as there are no tickets), right-click Remove, and an **Upgrade** button. The session's Tools menu offers Files and the two printing items.

H6. Upgrading, in place

Portal → **Upgrade** (or the console's Upgrade button): what the full version adds, the price — **free now, R500 a month after a 14-day free trial** — and the full terms. The owner ticks and presses Upgrade. The **same account** becomes a normal company: the same computers, the same saved passwords. A console that is open switches to the full console at once. The full terms and the price agreed are kept on the Billing page. The free computers keep working as they are; reinstalling one gives it the full version's tray app.

Where: `Platform/Pages/Portal/Upgrade.cshtml` , `Services/TenantService.cs` (`UpgradeAsync`), `Hub/TenantHub.Free.cs` (`PlanChangedAsync`).

03 — How the security works

A remote support tool can control customers' computers, so security matters more here than in most software. This document explains, in plain terms, what protects what, and is honest about what is not yet done.

The detailed engineering record is `HANDOVER.md` §72 (signing in), §73 (passwords), §31/§41 (unattended), §77 (unattended password) and §76 (permission prompts); since V2 also §100 (remembered unattended passwords), §101 and §105 (the free version's limits), §104 (*Being controlled remotely*) and §106 (removing machines, and Uninstall).

The one idea to hold on to

The thing that guards every customer computer is a technician's sign-in. A technician who is signed in can reach every machine of their company that allows unattended access, and scripts run there with full system rights. So most of the security below is about making sure **only the real technician can sign in**, that **access stops immediately** when it should, and that **the customer stays in control** of their own machine.

---

1. Everything is encrypted on the way

- All traffic between every program and the server uses **TLS 1.3** — the same protection as online banking. It was measured on the live server on 1 October 2026: TLS 1.3, with a key exchange that is designed to resist future quantum computers (`X25519MLKEM768`), and AES encryption.
- The certificate comes from **Let's Encrypt** and is renewed automatically by **Caddy**, the web server that sits in front of the platform.
- **The programs refuse to talk unencrypted over the internet.** If a program is ever pointed at a plain `http://` address on the public internet by mistake, it refuses to connect. Unencrypted is only allowed on a private office network (for testing). This rule is in `Shared/SecureTransport.cs` .
- None of the programs ever switches off certificate checking.

And end to end, since 7 October 2026 (HANDOVER §112, not yet seen on a real machine). TLS ends at the server, so before this a session was readable INSIDE the server — the picture, every key typed (passwords included), the clipboard, files, sound, scripts. Now the customer's machine and each technician's console agree a key the server never sees, and everything a session carries travels sealed with AES-256-GCM; the server relays it and cannot open it. Each machine has an identity key (made on the machine, readable only by the system) that signs the session key, and every console remembers it: if it ever changes — a reinstall, or somebody pretending — the console stops and asks before showing or sending anything. What the server still reads, on purpose: who connected to what and when, and the audit lines for the ticket (file names moved, a script's first line, reboots). Honest limit: a server that had been taken over could still START a session of its

own; it could not read or hijack a real technician's. The session window says which it is — a green *End-to-end encrypted*, or *Not end-to-end encrypted* when an older app is involved. The design is `docs/NEXT-SESSION-E2E.md`.

2. Signing in

Passwords

- Stored only as a **one-way scrambled value** (PBKDF2-SHA256, 600,000 rounds since 7 October 2026 — older ones are re-made at each person's next sign-in — a different random "salt" for each person). Nobody — not even someone with the database — can read a password back.
- At least 8 characters and not the person's email address.
- People **choose their own** passwords through an emailed link. Nobody types a password for someone else, so a supervisor never knows anyone's password.

Two-factor sign-in (2FA), for everybody

- After the password, a **six-digit code** from an authenticator app on their phone (Google Authenticator, Microsoft Authenticator, etc.). This is the standard called TOTP.
- Each code works **only once**.
- **Ten recovery codes** are given once at setup, for a lost phone. Each works once. They are stored only as a scrambled value.
- The code is asked **once every 7 days** on the same computer or browser; the password is asked every time. That "trusted for 7 days" token is tied to the person and is cancelled instantly by any password change, role change or sign-out.
- A supervisor can **reset** someone's two-factor (they set it up again at their next sign-in).

Stopping guessing

- 5 wrong attempts on one account → that account is refused for 15 minutes, even with the right password.
- 30 wrong attempts from one internet address → that address is refused.
- "Forgot password" gives the same answer whether or not the email exists (so it cannot be used to find out who has an account) and is limited to 3 emails an hour per address.
- A sign-in from a **new network** is emailed to the person.

Being signed out at once when it matters

- Every sign-in carries a **security stamp**. Changing a person's password, roles or two-factor, removing them, or pressing **Sign out** on the People page changes the stamp. Their console is **closed within a second**, and their web portal on the next page they open.
- A console's sign-in expires after 12 hours unused.

One sign-in at a time (added 2026-10-05)

- Each person can be signed in to **one console and one web browser** at a time. Signing in to the console on a second computer closes the first console; signing in on a second browser signs the first browser out on its next page. The person who was pushed off is told why.
- Signing in on a **different** computer asks for the two-factor code even if that computer was trusted, and cancels the 7-day trust on every computer, so the first one is asked for the code too. This is in case the second sign-in was somebody else who has the password.
- Signing in again on the **same** computer, or after pressing Exit or Sign out, asks for nothing extra. The console's Tickets button opens the portal without counting as a second browser.

Sign-in links

- Invitation links last 72 hours, password-reset links 2 hours, and each can be used **once**.
- A link is used up only when the new password is actually submitted, not when the email is opened (email scanners open every link).

The platform owner signs in with a long secret **owner key** and a separate authenticator code.

3. The customer stays in control

- **Permission before every attended session.** The customer sees who is connecting and must click Allow. No answer within the time limit means no. Until they say yes, the program on their machine connects but **does nothing at all** — no picture, no input.
- **Unattended access is the customer's switch**, in their tray app. Untick it and unattended access stops. Only the customer's own tray app can change it — not the server, not a technician.
- **An unattended password** can be set on the machine by the person there. Then nobody, including the IT company's owner, gets in unattended without it. There is no office override, on purpose. Five wrong tries lock the machine for 15 minutes.
- **Remembered for each technician (V2).** Once a technician has typed it correctly, it is not asked of *them* again. This loosens the rule above on purpose and only this far: every technician still has to type it once. The server keeps only *that it was typed right, against which password* — never the password or its scrambled form. Changing the password at the machine forgets it for everybody. Someone else guessing (and locking the machine) does not lock out somebody who already knows it.
- **Uninstall (V2).** The person at the machine can remove the product themselves from the tray app; Windows asks for administrator permission. The IT company sees that it was uninstalled.
- **Only the machine's own tray app speaks for the person at it.** The permission switch, the password, closing the app and uninstalling are all accepted by the server only from that connection — never from the agent service (which is what they guard) and never from a technician.
- **Exit** in the tray app takes the machine fully offline for support.
- **The microphone** is never opened without asking the customer in that session, and **never** in an unattended session. The server itself blocks any attempt by a technician's program to answer that question on the customer's behalf.

- The customer is **told** when a technician connects, when files move, and when a session is being recorded.
- The clipboard is shared **only while a technician is connected**.

4. What each person is allowed to do

- Roles are **Owner**, **Supervisor**, **Technician** and **Office administrator**, as tick boxes.
- Every permission check is done **on the server**, in one place (`Platform/Services/Permissions.cs`). Hiding a button is never relied on — the server refuses the action itself.
- Office administrators **cannot connect** to any customer machine.
- A technician never closes a ticket; "do not invoice" needs a supervisor.
- Only a supervisor or the owner can delete a session recording.
- Only a supervisor or the owner can **remove a machine** (V2), and it needs a reason. A removed install is refused if it ever comes back — it cannot quietly put itself back on the list. An online machine is told to uninstall itself; the instruction is sent only to its agent, only at that moment.
- Only a supervisor or the owner can see **who** has a machine's unattended password remembered, and clear it. Nobody ever sees the password.

4b. The free version (V2)

- Its limits — one person, two computers, unattended only, a password on every computer, and only control, files, copy and paste and printing — are **enforced by the server**, not by hiding buttons. The hub refuses a third computer when it connects and drops any session message that is not on the free version's **allow-list** (so a message added to the product later is refused to the free version until somebody decides otherwise). The portal has one rule in front of every page and API, also an allow-list.
- A free computer is installed with the password **typed at the computer**, and set by one connection made **as that computer**, so the agent service still never unlocks itself.
- With no tray app, the person at a free computer is told by the **"Being controlled remotely"** bar, for the whole of every session, naming who is connected.
- The free version's terms say it is for **the account holder's own computers only**.

5. Keeping companies apart

- Each IT company has its own separate hub in the server's memory, and every database query is filtered by company. One company cannot see another's customers, machines, tickets or sessions.
- **The tenant key is not a password.** It is a long random code (192 bits) that every customer's install needs, so it is public by design — it says "this machine belongs to Anywhere IT". It lets a machine *ask for help*; it never lets anybody *in*. Making it longer would change nothing. Do not try to "fix" it.

6. Customer machines

- Each customer computer gets its **own secret device key** on first connect. Another computer cannot pretend to be it without that key.
- A machine that is **reinstalled** is recognised by a scrambled hardware fingerprint, so its history and customer link are kept rather than it appearing as a new machine. Because any program on a computer can read its hardware fingerprint, **unattended access on a reinstalled machine waits until a supervisor or the owner confirms it is the same machine** (V2.1, `HANDOVER.md` §111).
- **Files received are checked** so they can never be written outside the folder intended. (Four real ways to break out of the folder were found by the test tools and closed.)
- The host runs with **full system rights**. That is normal for remote support tools and is what makes password prompts and administrator programs work — but it is a deliberate choice and is why the sign-in protections above matter so much. **Scripts also run with full system rights.**

7. Stored data

- **Workshop device passwords** and **email account passwords** are stored **encrypted with AES-256** using the server's key folder, `data/keys`. They are shown only to signed-in staff and only behind a Show button.
- **The key folder is critical.** Lose it and every stored password is unreadable. Never change the line `SetApplicationName("RemoteSupportPlatform")` in `Platform/Program.cs` — it is part of how those keys work, and changing it has the same effect as losing them.
- **The screen picture is never stored by the server.** It only passes through. The exception is a session recording, which a technician chooses to make and which is stored on the server in `data/recordings` and linked to the ticket.

8. The server itself

Set up automatically by `deploy/go-live.sh`:

- Security updates install themselves.
- **SSH (remote login to the server) uses a key only**; password login is switched off. Repeated failed logins are blocked by **fail2ban**.
- The platform runs as its own limited account (`rsp`), cannot change its own program files, and only it can read its data folder. Extra Linux sandboxing is switched on.
- The website sends browser security headers (HSTS so browsers always use https, no framing of pages except website chat, which must be framed to work).
- **Nightly backups** of the database and the data folder, encrypted with AES-256, kept for 14 days in `/var/backups/rsp`. The passphrase is `/etc/rsp-backup-key` — **without it the backups cannot be opened**, so it must be copied somewhere safe (see document 08). An office PC can fetch each night's copy through an account that can do nothing but list and fetch those encrypted files (`rspcopy`).
- **The database (PostgreSQL, since 9 October 2026) has no password and listens on no network port**: only the platform's own Linux account can reach it, over a local socket.

- **Caddy reaches the platform through a Unix socket** that only Caddy and the platform can open; the platform trusts the visitor's address Caddy passes on only when it arrives that way (`UnixSocket.TrustCaddy`).

8b. The people who run the platform (V2.2)

- **The owner key opens nothing on its own.** `/owner` needs the key **and** the owner's authenticator code. Two older addresses that answered the key alone were removed on 15 October 2026, and `tools/securitycheck` fails if they come back.
- **Platform admins** sign in to `/owner` with their own email, password and authenticator, at one of four levels (Viewer, Manager, Full, Owner). Every change they make is logged by name, and **every change to a company goes through one place** (`AdminCompanies`). An Owner-level admin can never be removed from the pages. Suspending a company now takes effect at once: its consoles and machines are turned away and its people signed out.
- **Keys are never printed.** The set-up script says where each key is kept, never the key. If one is ever exposed, `deploy\Rotate-Keys.ps1` replaces the owner key and the server key — done on 15 October 2026 after a log containing both was pasted into a chat; the keys in it are dead.
- **Servers talk to each other with a server key**, never an admin's password (the control dashboard).

8c. Payments (V2.2)

- **No card number ever reaches our server.** The customer types it on PayFast's own pages; we keep only PayFast's token for the subscription.
- A payment notice is believed only if it comes **from PayFast's own addresses**, carries a valid signature, names our merchant account and our subscription, **and** PayFast itself confirms it when asked back. Refused notices are not stored. The account details are put on the server with `deploy\Set-PayfastKeys.ps1`, never typed into chat or a command line.

8d. Updates (V2.3)

A program that installs other programs on customers' computers is exactly what an attacker would want to control, so **the server is never trusted to say what is safe — the signature is.** Each computer checks every file of an update itself, with Windows' own signature check, before anything runs:

- **Signed by us**: the certificate's organisation must be *The Remote Support Program (Pty) Ltd* in South Africa, read as fields of the certificate (a certificate anybody makes with our name in it is refused — the test tool proves it). Not "signed by anyone": any company can buy a certificate.
- **Valid and not revoked**: if the revocation list cannot be reached, the update waits and tries again.
- **Newer than what is installed**: so somebody who took over the server still could not push an old, genuinely signed program with a known fault. Only the computer's own rollback ever goes back.
- **There is no test mode** that relaxes any of this.

An update only ever comes from the computer's own IT company's server, and only an agent that says it can update itself is offered one. A release reaches our own company first and pauses itself if computers roll back. See HANDOVER §148.

8e. The outside security review (V2.4)

On **19 October 2026** an outside tester (Daniel) reviewed the live site from outside and signed in as a company owner. **They found no way into another company's data and no account takeover.** Every finding was checked against the code before anything was changed (HANDOVER **§158** — read it before "fixing" any of them again; four were already right and the report could not see why):

- **Fixed:** a stranger could keep a person locked out by typing five wrong passwords every 15 minutes. A computer holding that person's *trusted for a week* sign-in is now counted on its own, so a stranger's failures never lock it. The holding page now sends the same security headers as the platform. A full content-security policy is sent **in report-only mode** (45 pages still have script written into them; browsers report what it would block to `/api/csp-report`, and that log is the list to clear before it is enforced). A missing ticket or article is a plain 404. A stored picture must hold together as a picture, not only start like one. `/health` says only `{"ok":true}`. `/.well-known/security.txt` exists.
- **Already right:** a limit per address behind the per-account one; *Forgot password* is throttled but answers the same whatever happens (so nobody can use it to ask which addresses exist); the runtime is the patched one (it is packed inside the program — read the build's `deps.json`, not the server's `dotnet --list-runtimes`).
- **Addresses use the company's own numbers (§159).** The address bar used to show the database's row number, one sequence for every company, which let a company estimate the size of the whole platform. Now `/portal/t/12` is that company's ticket #12, clients and articles have their own numbers too, and stored files are reached by a random token (`/portal/p/…`). Every lookup is made **with the company in the same query** (`Services/PortalAddresses.cs`). Old addresses keep working for their own company and are a 404 for anybody else.
- **Still to do:** the tester's database held one company, so **access from one company to another was never tried**. Give them a second company and a technician login and have that re-run.

9. Honest list of what is NOT done yet

These were identified on 1 October 2026 and agreed as the order of work after going live.

1. ~~**Taking over a machine by its hardware fingerprint.**~~ **BUILT 7 October 2026 (HANDOVER §111), not yet on hardware.** A machine matched to its old record by hardware alone keeps the record, but unattended access waits until a supervisor or the owner confirms it is the same machine — the customer's own tick is not enough, because an impostor would hold that too.
2. ~~**Raise password scrambling**~~ **BUILT the same day (§111)**: 600,000 rounds, older passwords re-made at each person's next sign-in.

3. ~~**Code signing.**~~ **DONE 15 October 2026.** The certificate (Sectigo OV, through LeaderSSL, on a YubiKey) was issued on 14 October, and the first signed deploy went live the next day: every program and script the server hands out is signed and timestamped as The Remote Support Program (Pty) Ltd, and `Publish-Platform.ps1` refuses to upload anything unsigned. Programs installed on customers' computers before then stay unsigned until their one update by hand (V2.3, section 8d); after that they update themselves, signed. Windows' SmartScreen may still warn on a brand-new certificate until it has built up a reputation. See `docs/CODE-SIGNING.md`.
4. ~~**End-to-end encryption of sessions.**~~ **BUILT 7 October 2026 (HANDOVER §112), not yet on hardware** — see section 1 above.
5. ~~**A copy of the backups off the server.**~~ **BUILT 8 October 2026 (§120)**: an office PC fetches each night's encrypted backup. Proved on this PC against the real gatekeeper; **check it has been set up on the real server and office PC** (`deploy/backup/README.md`).
6. **An outside penetration test** before selling widely. **A first one was done on 19 October 2026** (section 8e); it still needs re-running with two companies, to try reaching one from the other.

Smaller known points:

- Sign-in lockouts are counted in memory, so a server restart clears them. A restart also signs every console out (by design).
- Nobody is told at the machine when a wrong unattended password is typed; only the server log has it.

10. Rules for whoever maintains this

- Never take a password in a chat, email or ticket. To store one for a script, use Windows' `Get-Credential | Export-Clixml`, which keeps it encrypted to that Windows user.
- Never switch off two-factor (`Security:RequireTwoFactor`) on a server customers can reach. It exists only for test tools that run their own throwaway server.
- Never commit secrets to the code repository. The `.gitignore` file lists what must stay out and why.
- Run `tools/securitycheck` after any change to sign-in. It has 78 checks and has proved, by removing each protection in turn, that it notices when one is missing. After a change to unattended passwords run `tools/unattendedpw`; to the free version, `tools/freecheck` and `tools/freeportal`; to removing machines, `tools/removecheck`.
- **Never put words from a web address on a page.** A result passed back in an address is a short code or an id, and the page chooses its own words — otherwise anyone could craft a link that shows a sentence of their choosing inside the portal (found and avoided while building the free version's Remove, `HANDOVER.md` §105).

04 — Where it runs

This is the map of every outside service the product depends on, as of 15 October 2026 (V2.2). Passwords and logins are **not** here — document 08 says where they are kept.

The authoritative technical records are `docs/HOSTING-DNS.md` (domain, DNS, mail) and `docs/GO-LIVE.md` (the server). If those and this page ever disagree, they win.

The suppliers

What	Supplier	Detail
The domain name `theremotesupportprogram.com`	**Afrihost** (registrar)	Only the registration is there. Renew it every year — if it lapses, every installed customer app stops working.
DNS (what the name points to)	**Cloudflare**	Nameservers `titan.ns.cloudflare.com` and `rosemary.ns.cloudflare.com`. **All DNS changes are made in Cloudflare**, not at Afrihost.
The live server	**xneelo Cloud** (OpenStack)	Ubuntu 24.04, 2 processors, 4 GB memory. A system disk plus **two volumes** added on 9 October: `rsp-files` (50 GB, the data folder and the backups) and `rsp-db` (20 GB, PostgreSQL). Public address **154.65.110.136**. A load test on a bigger server (4 processors, 8 GB) carried 18 companies of 10 technicians and 500 machines each (`HANDOVER.md` §126).
Email for the domain	**xneelo shared hosting** (konsoleH control panel)	Mailboxes such as `info@`. Separate login from xneelo Cloud. Mail stays here permanently, never on the platform server.
The website's security certificate	**Let's Encrypt**, through Caddy	Free, renewed automatically on the server. Nothing to do.
Code-signing certificate	**Sectigo**, bought through **LeaderSSL**	Ordered and paid 2 October 2026; **issued 14 October 2026, valid to 18 November 2027**, the first of the two-year term's two certificates. It is on the **YubiKey 5 NFC FIPS** token; **every**

What	Supplier	Detail
		program the server hands out has been signed with it since the 15 October deploy. See `docs/CODE-SIGNING.md` and `docs/CODE-SIGNING-ORDER.md`.
Card payments	**PayFast** (South Africa)	A merchant account with Recurring Billing. Built and proved on PayFast's **sandbox**; the live server is still on the sandbox account and the pages are hidden until the real website goes up. PayFast's Recurring Billing page sends notices to `https://theremotesupportprogram.com/api/payfast/notify` and allows only the live server's address — **move it if the server moves**. `docs/NEXT-SESSION-PAYFAST.md`.
Company registration	**CIPC**	The Remote Support Program (Pty) Ltd, 2026/737434/07

The live server

Address	`154.65.110.136` — the website is `https://theremotesupportprogram.com`
Login	`ssh ubuntu@154.65.110.136` — **SSH key only**, no password login. The private key is on Clint's laptop (`C:\Users\Clint\.ssh\id_ed25519`) and should also be in the password manager.
If SSH fails	The xneelo Cloud panel has a **Console** button on the instance's page — the way back in.
Program folder	`/opt/rsp`
Data folder	`/opt/rsp/data` — the encryption keys (`keys/`), logos, photos, files on comments, recordings, health check reports. Since 9 October it lives on the `rsp-files` volume, mounted back at the same path
Database	**PostgreSQL** on the `rsp-db` volume (database and role `rsp`, signed in over the local socket — **no database password exists**, and nothing listens on the network). The drop-in file `/etc/systemd/system/remotesupport-platform.service.d/postgres.conf` is the one sign the server is on PostgreSQL. The old SQLite file `platform.db` is kept frozen as it was at the switch; `deploy\postgres\Switch-BackToSqlite.ps1` is the way back (`HANDOVER.md` §125)

Service name	`remotesupport-platform` (runs as the Linux user `rsp`)
Web front	**Caddy**, on ports 80 and 443, passing traffic to the platform through the Unix socket `/run/rsp/platform.sock` (since 15 October; `127.0.0.1:8090` is kept for checks on the server itself)
Owner key	`/etc/rsp-owner-key` (the first and last way in to `/owner`, always with the owner's authenticator code; named **platform admins** sign in with their own email instead). Replaced with `deploy\Rotate-Keys.ps1` if it is ever exposed — it was, and was, on 15 October
Server key	In the service settings `go-live.sh` writes (`/etc/systemd/system/remotesupport-platform.service.d/override.conf`, `RSP_Platform__ServerKey`); lets the control dashboard on one server read and change companies on another. Replaced together with the owner key by `Rotate-Keys.ps1`
Backup passphrase	`/etc/rsp-backup-key`
Nightly backups	`/var/backups/rsp` (on `rsp-files`) — a `pg_dump` of the database plus the data folder, encrypted, 14 days kept; each finished session recording encrypted once on its own under `recordings/`. A backup is also taken automatically before each update. An **office PC can fetch a copy every night** through a locked-down account `rspcopy` (`deploy/backup/README.md`; built and tested, check it is running)
Firewall	Inbound ports 22 (SSH), 80 and 443 only — set in xneelo's *Security Groups*

Useful commands on the server (after `ssh ubuntu@154.65.110.136`):

```
sudo systemctl status remotesupport-platform        # is it running?
sudo journalctl -u remotesupport-platform -f        # watch its log live
sudo systemctl restart remotesupport-platform       # restart it (signs every console out)
cd /opt/rsp && sudo -u rsp ./RemoteSupport.Platform   # list the command-line commands
```

The command-line commands include: `create-tenant`, `invite`, `list-tenants`, `list-techs`, `set-brand`, `set-logo`, `remove-tech`, `reset-2fa`, `signout`, `reset-owner-2fa`, `platform-mail`. `set-brand <company> plan free` (or `full`) puts a company on the free version or takes it off — the way to try the free version while public signup is off. `pilot-link "<company>" [email]` makes a pilot company's one-use sign-up link (the same as **Owner → Pilots**); `set-brand <company> pilot true` makes an existing company a pilot.

The owner pages and the control dashboard

`https://theremotesupportprogram.com/owner` is where the platform itself is run: every company (approve, suspend, change), the platform mailbox, PayFast, pilot links, reviews waiting for approval, and since 13 October a **card per server** with its readings and how full it is (amber at 70%, red at 85%), emailed to the admins when it crosses a level. People sign in there as **platform admins** with their own email, password and authenticator, at one of four levels (Viewer, Manager, Full, Owner); everything they do is logged by name. The owner key plus the owner authenticator remain the first and last way in. Other servers, when there are any, are read with their own server key, never a password (`HANDOVER.md` §132–133).

What the public sees: the holding page

The public front of the website — the home, features and pricing addresses — is a **holding page** (`landing/index.html` in the code), on purpose, until Clint decides the real site is ready. It carries the company's details and the **terms, refund policy and privacy policy** that PayFast checks for. Everything else (portals, downloads, every app's connection, a pilot's `/pilot/…` link) reaches the platform as normal. **The real website is built** (twelve pages, light and dark, screenshots, company reviews — `docs/WEBSITE.md`) and sits behind the holding page: every one of its addresses is in `go-live.sh` 's holding list, and `Publish-Platform.ps1 -RealSite` is what shows it. A platform admin can see it on the live server at **Owner** → **Preview** without publishing it. **Public signup is switched off** on the live server too (`Platform:SignupsEnabled=false`), so the free version cannot be signed up for yet. Document 05 says how each is changed.

Going public on 1 November 2026. The deploy with `-RealSite` shows the real website, **opens signups** and puts **our chat bubble** on the site (answered by Anywhere IT's people). Since 27 October `go-live.sh` writes all three from the same choice on every deploy, so they cannot disagree — and **every deploy after 1 November must carry** `-RealSite` , or the holding page comes back and signups close (HANDOVER §169).

Updating the live server is one command from the build PC — see document 05. It backs up the data first and keeps it. Since 26 October a deploy restarts Caddy first and starts the platform after it, so every machine and console reconnects **once** per deploy, not twice; if anything between fails, the platform is started anyway (§165).

Knowing when it is down

The office PC watches the site (built 27 October, HANDOVER §170; **not installed yet** — one command, `deploy\monitor\Install-SiteWatch.ps1` , on the office PC that already copies the backups; `deploy/monitor/README.md`). Every 2 minutes it asks `/health` ; after **two misses in a row** it emails Clint **once** that the site is down, and once when it is back with how long it was down. Once a day it also warns if the website certificate has under 14 days left or the office's backup copy is more than 36 hours old. It asks the office's own internet first, so an office outage is not reported as the site being down — but while the office PC is off or offline, **nothing is watching**. Mail goes through a mailbox on xneelo's shared hosting, apart from the server it watches.

Restoring from a backup

Backups are `.tar.gz.gpg` files, encrypted with the passphrase in `/etc/rsp-backup-key`. Since the move to PostgreSQL each one holds a `pg_dump` **of the database** and a copy of the data folder (without recordings, which are backed up one by one). To look inside one on the server:

```
sudo mkdir -p /tmp/restore
sudo gpg --batch --passphrase-file /etc/rsp-backup-key -d /var/backups/rsp/rsp-YYYYMMDD.tar.gz.g
```

Restoring means stopping the platform, loading the dump into an empty `rsp` database, copying the data folder back with ownership `rsp:rsp`, and starting it. A dump restored into an empty database is proved to match the original row by row (`tools/backupcheck`), but **there is no one-command restore script yet**: read `deploy/rsp-backup.sh` (which makes the backup) first, and practise a restore once on a spare server.

If the server is lost completely: build a new one with `docs/GO-LIVE.md` (about an hour), point the DNS at it in Cloudflare, then restore the latest backup — which you can only do if a copy of the backup and the backup passphrase exist somewhere other than the lost server. That is why document 08 asks for both to be kept off the server.

DNS records (in Cloudflare)

Name	Points to	Purpose
`@` and `www`	154.65.110.136 (the live server)	The website, portals and every app's connection
`mail`, `smtp`, `webmail`	197.221.14.40 (xneelo shared hosting)	Email — separate on purpose, so moving the server never breaks mail
MX, SPF, DMARC	xneelo mail	Email delivery and anti-spoofing

All records are **"DNS only" (grey cloud)** in Cloudflare — not proxied. The full table and the records that were deliberately removed are in `docs/HOSTING-DNS.md`.

Email the product sends

The platform sends invitations, password resets, trial notices, new-sign-in alerts, health check reports, payment receipts and reminders, and quote requests. It uses a mailbox set in the **owner portal → Mail**, sending through `smtp.theremotesupportprogram.com` on port **587** (not 465). Each IT company can also set its own mailbox for its health check reports.

The office test setup (Anywhere IT, Knysna)

None of this is needed to run the live product, but it is where testing happens.

Build PC	Clint's laptop, `C:\Users\Clint\Desktop\Remote Support`. Also runs a **test copy of the platform** at `http://192.168.1.115:8090` (office network only).
Test machine 1	**DESKTOP-OPHQIDH** — a desktop with no hardware video compression; tests the fallback paths, file transfer, clipboard, unattended
Test machine 2	**DESKTOP-2N6FVGT** — a gaming laptop with NVIDIA graphics; tests picture quality and the codec
Shared drive	`\\192.168.1.33\anywhereit` (a WD My Cloud) — holds the **Test Checklist** program staff use to log test results
The original health check	`C:\PCHealthCheck` on the build PC — the standalone toolkit the product's health check came from. Its own project; the product's copy is in `healthcheck/`

05 — Building and releasing

For developers. How to get from the code to running programs, and how a change reaches the live server and customers' computers.

What you need on the build PC

Tool	Why
Windows 10 or 11	The customer and technician programs are Windows programs and use Windows-only features
.NET 10 SDK (`C:\Program Files\dotnet\dotnet.exe`; `winget install --id Microsoft.DotNet.SDK.10 --exact`)	Builds everything. The scripts expect it in that folder
Windows PowerShell 5.1 (built into Windows)	Every build script. Run them as `powershell -ExecutionPolicy Bypass -File <script>` — the plain form is blocked by Windows' script policy
Windows OpenSSH Client (Settings → Optional features)	Uploading to the live server
The SSH key for the live server	See document 08
Windows SDK (for `signtool.exe`), **Yubico's Smart Card Minidriver** (x64) and the **YubiKey** code-signing token	Only for signed builds
Inno Setup 6 (optional)	Only for the old-style setup .exe files (`build.ps1 -Installers`). The normal download no longer uses them

No other paid tools are needed. Two outside libraries are brought in, both MIT licence and free: **QRCoder**, for drawing the two-factor QR code, and **MailKit**, for reading and sending email for tickets by email (since October 2026). Everything else is .NET itself.

First time: get the code ready

1. Copy the `Source code` folder to the PC, or restore it with its history (see `Source code with history\HOW-TO-RESTORE.txt`).

2. Create `branding\brand.json` from `branding\brand.example.json`. For testing against your own local platform, set `serverUrl` to `ws://<this PC's address>:8090` and leave `tenantKey` empty until you have created a test company (the portal's Downloads page shows the real values).
3. Open `RemoteSupport.sln` in Visual Studio 2022 or use the command line.

Run the platform locally (for testing)

```
dotnet run -c Release --project src/RemoteSupport.Platform
```

It listens on port 8090 and creates its database and keys in its `data` folder on first run. Use its command line to create a company and a first owner:

```
src\RemoteSupport.Platform\bin\Release\net10.0\RemoteSupport.Platform.exe create-tenant "Test IT
src\RemoteSupport.Platform\bin\Release\net10.0\RemoteSupport.Platform.exe add-tech testit you@ex
```

(Run the exe with no arguments to see every command and its exact syntax.)

`tools/workflowdemo` creates a complete demonstration company with clients, staff and tickets on a throwaway platform, without touching anything real — the quickest way to see the portal working.

The build scripts

Command	What it does	When to use it
`build.ps1`	Builds every program into `dist\`	General build
`build.ps1 -Platform -PublicUrl https://theremotesupportprogram.com`	Also builds the Linux server package with the customer and technician downloads inside it	Called for you by `Publish-Platform.ps1`
`tools\ship-build.ps1`	A **full ship** to the office test platform: rebuilds the host, agent, tray app and console, refreshes the download, refreshes `demo\technician`	Any change to the customer side (`Host`, `Agent`, `Client`, `Shared`) — then reinstall the client on the test machine
`tools\ship-build.ps1 -`	Just the console, about 90 seconds, **to the office test platform only**	A change that touched only `RemoteSupport.Technician` (or the

Command	What it does	When to use it
`OnlyTechnician`		portal). **Live technicians get a new console from** `Publish-Platform.ps1` (the console rides inside the server package) and then re-download it — `HANDOVER.md` §110
`deploy\Publish-Platform.ps1 -Server 154.65.110.136`	**Puts the live server up to date.** Builds, uploads, backs up the data, installs, restarts, checks the site answers. The public front stays the holding page	Every live release
`deploy\Publish-Platform.ps1 -Server 154.65.110.136 -LandingOnly`	Uploads **only the holding page** (`landing\index.html`) — nothing built, shipped or restarted	A change to the holding page or its legal text
`deploy\Publish-Platform.ps1 -Server 154.65.110.136 -RealSite`	As the first, but shows the platform's own website to the public, **opens signups**, and puts our chat bubble on the site (`-SiteChat <company>` chooses whose chat answers it; Anywhere IT by default)	**From 1 November 2026, EVERY live release** — without it the holding page comes back and signups close (§169). Never before 1 November

Run them like this:

```
powershell -ExecutionPolicy Bypass -File deploy\Publish-Platform.ps1 -Server 154.65.110.136
```

Important habits, each learned the hard way:

- `demo\technician` and `demo\client` are ready-to-run copies **that go stale**. Refresh them (the ship script does) before testing, or you will test yesterday's program.
- If you are not sure whether a change touched the customer side, do a **full** ship. A stale host program looks exactly like a broken feature.
- A **platform restart signs every technician console out** (sign-in tokens live in memory). Warn people before updating the live server in working hours.
- **The terms** are in `src/RemoteSupport.Platform/Terms/` (one shared file, one per region, and the free version's section). After **any** change to them, move `Terms.Version` on and regenerate the holding page's legal text with `RemoteSupport.Platform landing-legal <full path to landing\index.html>`

— `tools/legalcheck` fails until you do. What a company accepted is kept word for word, so changing the files never changes what anybody agreed to.

- **Never use** `Publish-Landing.ps1 -Server` **on the live server** — it rewrites the web server's whole configuration and would cut the platform off. Use `Publish-Platform.ps1 -LandingOnly`.
- **Database changes** are made with Entity Framework migrations, applied automatically when the platform starts. **The live server is on PostgreSQL (since 9 October 2026) while local tests default to SQLite, so every model change needs TWO migrations, with the same name:** `dotnet-ef migrations add <Name> --project src\RemoteSupport.Platform --context PlatformDb -o Data/Migrations` and `dotnet-ef migrations add <Name> --project src\RemoteSupport.Platform --context PgPlatformDb --output-dir Data/PgMigrations`. `tools/pgcheck` fails until both exist. **Never re-make** `PgInitial` — it is frozen. Check a new SQLite migration's Designer file says `typeof(PlatformDb)`. Never switch to `EnsureCreated` — it silently ignores new columns.
- **Testing on PostgreSQL:** set `RSP_TEST_DB=postgres` (with the git-ignored `tools\rs-postgres.txt` naming a local PostgreSQL) and a harness runs on it; `tools\Run-DbHarnesses.ps1 -Postgres` runs them all. A new test tool that makes a database uses `UseHarnessDb`, `HarnessDb.MigrateAsync` and `HarnessDb.Use`.
- **Keys are never printed.** `go-live.sh` says where each key is, never the key itself. If a key is ever exposed (a log pasted somewhere), `deploy\Rotate-Keys.ps1 -Server <ip>` replaces the owner and server keys and puts the old ones back if the platform does not answer.
- **PayFast's account details** go onto the server with `deploy\Set-PayfastKeys.ps1` — never in chat, never as a command-line argument. Sandbox and live are separate accounts, changed together.
- **Any new public web page** must be added to `go-live.sh`'s holding list, or it is public the moment it deploys; `tools/sitecheck` fails otherwise.

How a release reaches people

What changed	What has to happen
The website or portal only	`Publish-Platform.ps1`. Done.
The technician console	`Publish-Platform.ps1`. An installed console (V2.3 or later) updates itself at its next start, or from **Update ready** in its header; an older one is downloaded again from `/tech/<company>` once
The customer side (tray app, agent, host, health check, the installer scripts)	`Publish-Platform.ps1`, signed. Computers that update themselves (V2.3 or later) take it by themselves when the release reaches their ring — see below. A computer installed **before 16 October 2026** needs its **one update by hand** first: connect, Tools → Scripts → *Update this machine*
A company's branding	Nothing to release: saving Portal → Branding reaches installed machines by itself (since 7 October)

Version numbers and rolling a release out (V2.3)

Every build is stamped with the moment it was made, in UTC — **2026.10.16.1432** is 14:32 on 16 October 2026 (`tools\BuildVersion.ps1` , used by `build.ps1` , `ship-build.ps1` and `Pack-ClientExe.ps1`). It only goes up, so the newest build is always the newest version. `build.ps1 -Version 2026.10.16.1500` stamps a chosen one, for testing. **Never pass** `-p:Version` **alone to a build**: `Directory.Build.props` pins the other version numbers too, so use all of `VersionArgs` (`HANDOVER.md` §147). A program built any other way says 1.0.0.

After a signed deploy, **Owner** → **Updates** shows the new release offered to **our own company only**. Our computers take it within minutes of being idle. **When they have come through it** (each shows *Updated*), press **Offer it to the pilots**, then **Offer it to everybody**. **Pause** stops it at once; it also pauses by itself if a computer rolls back, and emails every Full admin and Owner — read what that computer said before resuming. Each server's release is moved on separately, from the same page.

Code signing

The certificate is on the YubiKey (issued 14 October 2026, valid to 18 November 2027). The build signs automatically when the YubiKey is plugged in, using `tools\Signing.ps1` , and says plainly when it is producing unsigned files. **A PC that signs needs the Windows SDK and Yubico's Smart Card Minidriver** (x64) — without the driver Windows cannot use the key, which is ECC. Setting up a new signing PC, or loading the term's second certificate: `tools\Install-SigningCert.ps1` . Rules:

- Use `build.ps1 -RequireSigning` for anything going to customers, so it fails rather than shipping unsigned. `deploy\Publish-Platform.ps1` **does this itself since 15 October 2026**: it builds with `-RequireSigning` , then checks that every program and script the server hands out is signed by us, and uploads nothing otherwise. `-Unsigned` is the deliberate way past it.
- **Never sign test builds.** Sign only what is about to go to real customers.
- The one downloadable .exe is signed **inside first** (the programs packed into it), **outside last** — `tools\Pack-ClientExe.ps1` does this.
- The certificate is in the name **The Remote Support Program (Pty) Ltd**, never an IT company's name, because every reseller ships the same files.

`docs/CODE-SIGNING.md` and `docs/CODE-SIGNING-ORDER.md` have the full story. **The first signed deploy went live on 15 October 2026**: all 12 programs and scripts the server hands out, signed and timestamped.

The pilot guide

`docs/pilot-brochure/brochure.html` is the guide for pilot companies: setting up, what is new, and the full tour, with screenshots of made-up demonstration companies (never a real customer). It is printed to `The Remote Support Program - Pilot Guide.pdf` in the same folder with Edge's headless print-to-PDF. The portal screenshots come from a throwaway platform made by `tools/workflowdemo` ; the console and

customer-app windows are the real windows fed made-up data, the way `tools/siteshots` and `tools/lookprobe` do it.

Testing

Two kinds of tests exist, and both matter:

1. **The test tools in** `tools\` — about 150 small programs, each proving one feature. Most start their own throwaway platform, so they never touch real data. Each one prints how many of its checks passed (e.g. `securitycheck 75 of 75`). `HANDOVER.md` §7 explains them, and every section of the handover names the tools that prove that feature and the count they should give. Run them with, for example: `dotnet run -c Release --project tools/securitycheck`
2. **Hardware testing by people.** `docs/TEST-3.6.md` is the numbered checklist (722 checks). Staff work through it with the **Test Checklist** program (`tools/testchecklist`), published to the office shared drive, and export a daily report. **Never renumber a check** — the number is how results are matched up.

The working rule since day one: measure before changing code. When something is reported broken, first prove where it breaks (the logs and test tools are there for that), then fix it, then show the test that would have caught it fails without the fix.

Logs worth knowing

Log	Where	What it tells you
Console trace	Technician PC: `%LOCALAPPDATA%\TheRemoteSupportProgram\technician-trace.log`	Connections, messages lost, input that was blocked and why
Host trace	Customer PC: `C:\ProgramData\RemoteSupport\host-trace.log`	What account the host ran as, which screen it could see, desktop switches
Health check	Customer PC: `C:\ProgramData\RemoteSupport\healthcheck-agent.log`	Installing and removing the health check
Server	Live server: `sudo journalctl -u remotesupport-platform`	Everything on the server side, including security warnings at start-up
Agent state	Run `tools\agentstate.ps1` on a customer PC	Why unattended access is or is not working on that machine

06 — Map of the code

For developers. Where everything is, and the files you will most likely need first.

Language: **C# on .NET 10**. Windows programs use **WinForms**. The server is **ASP.NET Core** with **Razor Pages** for the website, **WebSockets** for live connections, and **Entity Framework Core** on **PostgreSQL** for the database on the live server (since 9 October 2026; **SQLite** for local testing and as the way back). Scripts are **PowerShell 5.1** (Windows) and **bash** (the Linux server). About 70,000 lines of hand-written code in `src` (not counting the database migrations, which are generated), plus the test tools.

Top-level folders

Folder	What is in it
`src\`	The product: the nine projects below
`tools\`	About 200 test and diagnostic programs, plus the ship, packing and signing scripts. Not shipped to customers
`docs\`	All technical documentation. `HANDOVER.md` is the master record
`deploy\`	Putting the server live: `Publish-Platform.ps1` (run on the build PC), `go-live.sh` (runs on the server), the systemd service file, `rsp-backup.sh` (the nightly backup), `Rotate-Keys.ps1`, `Set-PayfastKeys.ps1`; `postgres\` (the move to PostgreSQL and the way back), `backup\` (the office copy of the backups), `monitor\` (the office PC's check that the site is up, V2.4), `loadtest\`
`docs\pilot-brochure\`	The pilot guide for IT companies (`brochure.html`, its screenshots, and the printed PDF)
`docs\brochure\`	The general guide, for anybody thinking of using or investing (`brochure.html` → `The Remote Support Program - Guide.pdf` by `Print-Guide.ps1`)
`docs\logo\`	The logo and the mark, light and dark, with the source files
`deploy-client\`	The customer installer scripts: `Install-Client.ps1`, `Uninstall-Client.ps1` and double-click wrappers

Folder	What is in it
`healthcheck\`	The monthly health check toolkit (PowerShell), with its own README and decisions
`branding\`	Anywhere IT's branding for local testing. `brand.example.json` shows the format
`installers\`	Old-style Inno Setup installer scripts (`.iss`). No longer the main download
`landing\`	The holding web page used before the platform went live
`pitch\`	The investor presentation
`RemoteSupport.sln`	The Visual Studio solution with every project
`build.ps1`	The main build script
`Directory.Build.props`	Publisher name for every program, and the version a program built by hand says (1.0.0). A real build is stamped by `tools\BuildVersion.ps1` (V2.3)
`CLAUDE.md`	A short index of the project state, kept up to date after each piece of work

---

The projects in `src\`

`RemoteSupport.Platform` — the server (the biggest)

Folder or file	What it does
`Program.cs`	Start-up: settings, security headers, sign-in, web routes, WebSocket endpoints, and the **command-line commands** (`create-tenant` , `invite` , …) at the bottom
`Hub\TenantHub.cs`	**The heart of live support.** One per IT company: who is online, ringing, starting and ending sessions, relaying, chat, unattended permission and password. `Recompute` is the one place that decides if a machine is online/unattended

Folder or file	What it does
`Hub\TenantHub.Web.cs`, `TenantHub.WebChat.cs`	Browser support and website chat
`Hub\TenantHub.Outsiders.cs`, `GuestVisitor.cs`	Outside help: codes, visitors from another company, guests
`Hub\TenantHub.Watch.cs`, `TenantHub.Wake.cs`	Processes and Performance without connecting; Wake-on-LAN
`Hub\TenantHub.SavedPasswords.cs`	Remembered unattended passwords, per technician per machine (V2)
`Hub\TenantHub.Free.cs`	**The free version's rules** at the hub (two computers, unattended only, the session and console allow-lists), **removing a machine** for both versions, and the **upgrade** (V2)
`Hub\TenantHub.E2e.cs`	**End-to-end encryption** at the hub: says whether a session is sealed or clear, and refuses what must stay readable if it arrives sealed (V2.1)
`Hub\TenantHub.Branding.cs`	Sending a company's new look to its installed machines (V2.1)
`Hub\TenantHub.SessionLimit.cs`, `TenantHub.Bridge.cs`	Two sessions at once at most, and **copying straight from one session's machine to the other's** (`BridgeProblem` is the one rule) (V2.2)
`Hub\TenantHub.Update.cs`, `Services\UpdateService.cs`, `Services\UpdateSweep.cs`, `Pages\Owner\Updates.cshtml`	**Automatic updates** (V2.3): the release this server hands out (`/update/client.exe`, `/update/console.exe`), its rings and Pause, offering it to a few computers a minute, and what each computer reported

Folder or file	What it does
`Hub\HubRegistry.cs`	Finds the right company's hub
`Data\PlatformDb.cs`	The database: every table
`Data\Migrations\`, `Data\PgMigrations\`	Every change ever made to the database, for SQLite and for PostgreSQL (`PgPlatformDb.cs`), applied automatically. **Each change needs one in each** (document 05)
`Services\UnixSocket.cs`	How Caddy reaches the platform on the live server (`/run/rsp/platform.sock`), and trusting Caddy's forwarded addresses through it (V2.2)
`Services\Workflow.cs`	**Every ticket route and stage**, in one place
`Services\JobTypes.cs`, `Pages\Portal\JobTypes.cshtml`, `Hub\TenantHub.JobTypes.cs`	**Job types**: which categories and steps a company has switched off, the check that no route can break, the page, and telling open consoles (27 October)
`Services\TicketService*.cs`	Tickets: creating, moving between stages (`ActAsync` is the only thing that moves one), mentions
`Services\Permissions.cs`	**Who may do what.** Every permission check asks this
`Services\AuthService.cs`, `SignInGuard.cs`, `WebSignIn.cs`, `OwnerTwoFactor.cs`, `AccountLinks.cs`, `Passwords.cs`	Signing in, two-factor, lockouts, invitations and resets, password hashing
`Services\ClientService.cs`, `ContactService.c`	Customers, their people, importing a client list. `ClientService.Remove.cs` (V2.4): **removing a client**, everything on it moved to the right one first

Folder or file	What it does
`s`, `ClientImport.cs`	
`Services\PortalAddresses.cs`	**Every portal address** names things by the company's own number (`/portal/t/12`) or a file's random token, always looked up with the company in the same query (V2.4, §159)
`Services\TicketActivityService.cs`	**Latest activity** — every update on the tickets, newest first; who sees what is `TicketService.VisibleTo` (V2.4)
`Services\CompanyExport.cs`	**A company's data as one zip**, made in the background by `CompanyExportSweep` in the same file (Portal → Export, or ours for a closed company) (V2.4)
`Services\SiteChat.cs`, `CspReports.cs`	Our own site's chat bubble saying our name; the report-only content-security policy and its reports (V2.4)
`Hub\TenantHub.Invite.cs`	**A technician asking for outside help** in a session (V2.4, §155)
`Services\QuoteService.cs`	Quote requests, suppliers, items
`Services\HealthCheckService.cs`, `ReportPdfService.cs`	Receiving health check reports, making PDFs, emailing them
`Services\PayloadService.cs`	Builds each company's downloads (same files, their branding)
`Services\PlatformMailService.cs`	Sending email
`Services\StaffChatService.cs`, `WebChatService.cs`	Office chat and website chat
`Services\RecordingService.cs`,	Session recordings and ticket photos

Folder or file	What it does
`TicketPhotoService.cs`	
`Services\TicketIntake.cs`, `MailWire.cs`, `KnowledgeBaseService.cs`, `KbText.cs`	Tickets by email, and the knowledge base
`Services\Pricing.cs`, `Terms.cs`, `CompanyClock.cs`	Prices and seats, the signup terms, every time in the company's own time zone
`Services\FreePortal.cs`	The free version's portal: the **one** list of pages it may open (V2)
`Services\RecurringTaskService.cs`	**Recurring tasks**: every rule, and opening each due ticket at 06:00 company time (V2.1)
`Services\TicketService.Edit.cs`, `TicketService.Back.cs`, `TicketAttachmentService.cs`, `ClientUpdates.cs`	Editing a ticket, **going back a step** (every stage move writes a snapshot so it can be undone), files and photos on comments, emailing the client from a ticket (V2.2)
`Services\PayfastWire.cs`, `PayfastService.cs`, `CompanyBilling.cs`, `CompanyBilling.Late.cs`	**PayFast**: the signatures, checkouts and notices; switching a company on when it pays; keeping the amount true; late payments, reminders, suspension and cancelling (V2.2)
`Services\PilotService.cs`	**Pilot companies**: the one-use link and every rule (V2.2)
`Services\PlatformAdmins.cs`, `AdminCompanies.c`	**Platform admins** and their levels; every change to a company goes through `AdminCompanies`; the **control dashboard** reading other servers (V2.2)

Folder or file	What it does
`s`, `ControlServers.cs`, `ControlSweep.cs`	
`Services\SitePages.cs`, `SitePreview.cs`, `ReviewService.cs`	The real public website (hidden behind the holding page), previewing it, and companies' reviews (V2.2)
`Terms\`	The terms text: shared, one per region, and the free version's section
`Pages\`	The website: public pages, `Portal\` (office staff), `Owner\` (platform owner and admins), `Account\` (invitation and reset links). V2 added `Portal\Machines` (the free version's computers), `Portal\Upgrade` and `Portal\RemoveDevice`; V2.1 `Portal\Recurring`; V2.2 `Pilot` (a pilot company's sign-up), `SignupPaid`, `Portal\Review`, `Portal\_EditTicket`, `Portal\_Attach`, and the owner's Admins, Pilots, PayFast and server pages. `Pages\Portal\TicketWorkAttribute` is the **allow-list** of ticket-page actions an owner-only login may use
`Pages\Portal\_Timeline.cshtml`	How a ticket's "story" is drawn
`wwwroot\`	Browser files: `site.css` (all styling), and scripts for client search, phone layout, photos, chat, browser support, website chat; V2.1 `sounds.js` (the ding and ring) and `e2e.js` (end-to-end encryption in the browser — its list of readable messages must equal `Shared/SessionSeal.cs`'s, which `tools/e2ecore` checks)
`appsettings.json`	Settings. On the live server these are overridden by environment variables (`RSP_...`) in the service file and `appsettings.live.json`

`RemoteSupport.Technician` — the console

File	What it does
`Program.cs`, `LoginForm.cs`, `TwoFactorDialog.cs`, `TrustedSignIn.cs`	Starting up and signing in
`DashboardForm.cs`, `DeviceList.cs`	The main window and the machine list
`TechLink.cs`	The console's connection to the server

File	What it does
`SessionForm.cs`, `SessionForm.Voice.cs`, `SessionLink.cs`	The session window, its toolbar and tools, and reconnecting
`SessionForm.Between.cs`, `SessionForm.Bridge.cs`	Copy in one session window, paste in the other: through this PC, or straight between the two machines (V2.2)
`ScreenView.cs`	Drawing the customer's screen and sending mouse and keyboard
`ScreenView.Shortcuts.cs`	Sending Windows shortcuts (Win, Alt+Tab, …) to the customer while the picture has the focus (V2.1)
`IncomingCallPopup.cs`	The centre-screen call alert
`TicketsForm.cs`, `NewTicketDialog.cs`	Tickets in the console
`DragOut.cs`, `SessionRecorder.cs`, `AudioPlayer.cs`	Dragging files out, recording, sound
`StaffChatForm.cs`, `WebChatForm.cs`	Office chat, website chat
`SessionForm.Print.cs`, `MachineWatchForm.cs`	Remote printing; Processes and Performance
`CodeEntryDialog.cs`, `VisitorPopup.cs`, `GuestForm.cs`	Outside help: connecting with a code, watching a visitor, the guest console
`RemoveDeviceDialog.cs`, `ReasonDialog.cs`	Removing a machine with a reason (V2); why you are connecting, and the unattended password
`ConsoleDownload.cs`	The single-.exe console setting itself up
`ConsoleUpdate.cs`	The console updating itself before its sign-in window, and putting the old one back if the new one does not start (V2.3)
`Trace.cs`	The diagnostic log

`RemoteSupport.Host` — does the work on the customer's computer

File	What it does
`HostEngine.cs`	Runs the session: separate threads for capture, pointer and input
`DxgiCapture.cs`, `ScreenCapture.cs`, `LinkController.cs`	Screen capture (fast and fallback) and automatic quality
`StillRefiner.cs`	Sharpening a still screen after a change, so the picture never stays blurry (V2.2)
`HostEngine.Bridge.cs`	Sending or receiving copied files straight from another session's machine (V2.2)
`InputInjector.cs`, `DesktopFollower.cs`	Mouse and keyboard, and following Windows onto secure screens
`ConsentPrompt.cs`	The customer's permission prompt
`ClipboardAccess.cs`, `FileTransfer.cs`, `ExplorerSelection.cs`, `UserProcess.cs`	Clipboard, files, and the helper that runs as the logged-in person
`ScriptRunner.cs`	Running scripts
`AudioCapture.cs`, `AudioDiagnostics.cs`, `VoiceCall.cs`	Sound and voice
`PrivacyScreen.cs`	Blanking the customer's monitors
`ControlledNotice.cs`	The free version's *Being controlled remotely* bar (V2)
`PrintRedirect.cs`	Adding the technician's printer, and printing their file on the customer's printer
`UserContext.cs`	Finding the logged-in person's name and folders (the host runs as the system)
`MachineIdentityStore.cs`	The computer's own identity key for end-to-end encryption, readable only by the system (V2.1)

`RemoteSupport.Client` — the customer's tray app and the downloadable .exe

`MainForm.cs` (the window and tray icon), `RequestDialog.cs` (asking for help), `ChatPopup.cs`, `SessionBanner.cs` (the corner strip), `UnattendedPasswordDialog.cs`, `MicQuestion.cs`, `HostSpawner.cs` (starting a host), `Launcher.cs` + `LauncherForm.cs` + `ClientPayload.cs` (the one downloadable .exe: Get

help now / Install), `FreeInstallForm.cs` (the free version's *Add this computer*, V2), `OutsiderCodeDialog.cs` (a code for outside help). Uninstall… is in `MainForm.cs`.

`RemoteSupport.Agent` — the background service for unattended access

`AgentService.cs` (the service, and `UninstallSelf` when its machine is removed), `SessionLauncher.cs` (starting the host with system rights inside the logged-in session — Windows services cannot see the screen themselves), `HealthCheckKeeper.cs` (installing and removing the health check). Since V2.3: `AgentUpdate.cs` (taking an update, and `--apply-update`, the short-lived process that swaps the files, restarts the service and rolls back).

`RemoteSupport.Shared` — used by everything

`Protocol.cs` (**every message the programs exchange**), `WsConnection.cs` (the connection), `SecureTransport.cs` (refuses unencrypted connections over the internet), `Totp.cs` (two-factor codes), `DeviceIdentity.cs`, `MachineFingerprint.cs`, `FileSetTransfer.cs` (moving folders safely), `ClipLedger.cs`, `DownloadIdentity.cs`, `SystemSpecCollector.cs`, `Branding.cs`, `WakeOnLan.cs`, `MachineWatcher.cs` (`OutsiderAccess`, the outside-help rule, is inside `Protocol.cs`). Since V2: `FreePlan.cs` (every free-version limit, and `RemovalReasons`), `FreeSetup.cs` (setting up a free computer), `ClientUninstall.cs` (running the uninstaller, shared by the agent and the tray app). Since V2.1: `SessionSeal.cs` (end-to-end encryption: the seal, the handshake `E2eHandshake`, and the list of messages the server must still read), `MachinePins.cs` (each console's memory of every computer's identity), `BrandRefresh.cs` (a company's new look on an installed machine), `Chimes.cs` (the ding and ring, made in code). Since V2.2: `SessionLimit.cs` (two sessions at once), `BridgeSeal.cs` (sealing files sent between two machines), `TicketSort.cs` (the ticket list's orders), `JobParts.cs` (a job waits for its linked parts), `SupportRequest.cs` (a request's subject and description). Since V2.3, automatic updates: `AppVersion.cs` (version numbers), `UpdateTrust.cs` (the signature check every update passes), `UpdateRunner.cs` (download, check, wait, hand over; and the apply step), `UpdateSwap.cs` (swapping files and putting them back), `UpdateByHand.cs` (the script for the one update by hand).

`RemoteSupport.Media` — video and sound

`H264Encoder.cs` / `H264Decoder.cs` (Windows Media Foundation), `GpuColourConverter.cs`, `Nv12.cs`, `ScreenRecorder.cs` (writing .mp4), `PcmPlayer.cs`, `EchoCancelledMic.cs`.

`RemoteSupport.UI` — the look of the Windows programs

`Theme.cs` (colours, light and dark), `Controls.cs` (rounded buttons, cards, inputs), `BrandIcon.cs` (making icon files from a company's logo, and — V2.1 — making it every console window's icon), `WindowPlacement.cs` (remembering window positions), `GroupHeaders.cs`, `Sounds.cs` (playing the ding and ring, and muting them, V2.1).

`RemoteSupport.Server` — the old single-company server

Kept in the solution but **not used live**. Superseded by the Platform.

Where to start when…

You need to…	Start at
Understand a message between programs	`Shared/Protocol.cs`, then search for the message name in `TenantHub.cs`
Change a ticket route	`Platform/Services/Workflow.cs`, then run `tools/workflowcheck`, `tools/routecheck` and `tools/jobtypecheck` (which walks every route with every combination of Job types switches)
Change who can do what	`Platform/Services/Permissions.cs`, then run `tools/rolecheck`
Change sign-in	`Platform/Services/AuthService.cs`, then run `tools/securitycheck`
Fix something on the customer's screen side	`Host/HostEngine.cs`, and read `HANDOVER.md` §4 first (Windows permission levels and secure screens)
Add a control to a Windows window	Read `HANDOVER.md` §30 first: measure heights from the font, never type a number
Change the database	Edit `Data/PlatformDb.cs` and add a migration (document 05)
Change what the free version may do	`Shared/FreePlan.cs` (the limits), `Platform/Hub/TenantHub.Free.cs` (the hub), `Platform/Services/FreePortal.cs` (the portal), then run `tools/freecheck`, `tools/freeportal`, `tools/freeinstall`, `tools/upgradecheck`
Change removing a machine	`TenantHub.Free.cs` (`RemoveDeviceAsync`, `RemoveCoreAsync`), then run `tools/removecheck`
Add or change a message a session sends	Read `HANDOVER.md` §112 first. Ask `BinKind.Of(b[0])`, never `b[0]` (a sealed frame sets the top bit); a message the server must read goes in the clear list in **both** `Shared/SessionSeal.cs` and `wwwroot/e2e.js`. Then run `tools/e2ecore` and `tools/e2ewire`
Change anything that lets a technician in unattended	Ask `TenantHub.MayGoUnattended`, never the permission alone (a reinstalled machine waits for the office, §111). Then run `tools/reinstallcheck` and `tools/unattendedpw`

You need to…	Start at
Add a new way to move a ticket's stage	It must write `TicketNote.Before` (the snapshot that Back puts back) — read `HANDOVER.md` §128; run `tools/ticketstepcheck`
Add an action to a ticket page	Name it in `TicketWorkAttribute` only if an owner-only login may use it (§146); run `tools/rolecheck`
Touch payments	Read `docs/NEXT-SESSION-PAYFAST.md` and `HANDOVER.md` §134–137; run `tools/payfastcheck` and `tools/signuppaycheck` on both databases
Add a public web page	Add it to `go-live.sh`'s holding list; run `tools/sitecheck`
Change automatic updates	Read `HANDOVER.md` §147–§148 first. The signature rule is `Shared/UpdateTrust.cs` — never add a way round it; the rollout is `Platform/Services/UpdateService.cs`. Run `tools/updatetrust`, `updatewire` (both databases), `updateapply`, `consoleupdate`, `updatebyhand`, `versioncheck`
Add a file customers' computers must get	It reaches them in an update only if it is in the packed customer app AND `UpdateSwap.TopFiles` (or is a `healthcheck\*.ps1`); it must be signed

Reading the engineering record

`docs/HANDOVER.md` has 146 numbered sections, mostly in date order. Each one says what was asked for (often in Clint's own words), what was built, **what went wrong on the way**, which test tool proves it and what count it should give, and what still needs doing. When a section says it supersedes an older one, the newer one wins. It is long but it is the fastest way to avoid repeating a mistake that has already cost a day.

07 — Where things stand (28 October 2026, V2.5 — the final version before going live)

In one paragraph

The **remote control** half of the product is built and **proven on real computers**: screen, mouse and keyboard, secure screens, files and folders, copy and paste both ways, sound, reconnecting, restarting, and modern video compression. The **business** half — clients, tickets, job routes for every category, quotes, photos, roles, invitations, two-factor sign-in, office chat, website chat, browser support, tickets by email, a knowledge base — is **built and passes its automatic tests**, but much of it was written in the last four weeks and **has not yet been tried by staff on real computers**. Between 2 and 6 October a **free version** was added in full (document 10); on 7 October **sessions became encrypted end to end** (document 11). Between 8 and 15 October (document 12) the live server **moved to PostgreSQL** and was **load-tested** (18 companies per server), **PayFast** card payments, the **real website** and **company reviews** were built (all still hidden behind the holding page), **platform admins and a control dashboard** were added, **code signing went into use** (every program the server hands out is signed), everything moved to **.NET 10**, and a **pilot-company** offer was built — plus damage reports, files on comments, going back a step, editing tickets, two sessions at once with copy and paste straight between them, a sharper picture and handing over a website chat. On 16 October (document 13) **every program learned to keep itself up to date** — customer computers silently, in rings, with a rollback — which means that from the next release on, a fix reaches customers without anybody visiting them (once each computer has had its one update by hand). Since 25 October there are **three plans** (`HANDOVER.md` §161–§163, `docs/PRICING.md`): the **Free Version** (one computer, unattended with its password, nothing to pay), **The Basic Version for Remote Workers** (R50 a month, 3 to 20 computers, attended or unattended) and the full version; Free moves up to either, Basic up to full or down to Free. Between 17 and 27 October (document 14) came **job types**, **latest activity**, sorting and due dates, a technician asking for **outside help**, **exporting a company's data**, the fixes from an **outside security review**, per-company numbers in addresses, **removing a client**, and the website made ready to go public. On 28 October (document 15) Job types was made to show itself to an owner once only, as intended. The live server went up on **1 October 2026** with only Anywhere IT on it; public signup is off and the public front is a holding page **until 1 November 2026**, when the real website, signups and live PayFast payments go on together. Internal testing by Anywhere IT staff is under way. **The next job is proving, not building.**

Stage by stage

The full plan is `docs/ROADMAP-PARITY.md` (technical) and `docs/ROADMAP-PLAIN.md` (plain English, written 22 September — parts of it are now out of date; this page is newer).

Stage	What it is	State
1	Core remote control	**Done and proven.** Session recording, privacy screen, remote printing (both ways) and Wake-on-LAN also built (not yet on hardware). **Two sessions at once, with copy on one machine and paste on the other straight between them** (15 October), not on hardware
2	Picture quality: fast capture, H.264, automatic quality, real pointer	**Done and proven. A picture that never stays blurry** (15 October: key frames a minute apart, a still screen sharpened) built, not on hardware. Two loose ends: nobody has watched automatic quality on a genuinely bad line, and the NVIDIA chip on one gaming laptop refuses to compress (it falls back and still works)
3	Sound	Listening **proven**. Talking both ways **built**, not on hardware
3.5	Simplicity of the console	**All built.** Copy and paste proven by hand
3.6	Dashboard, ticketing, roles, photos, light/dark	**All built**, mostly not on hardware. 13 October: **edit a ticket, sort by date, damage reports, files and photos on comments, go back a step** — walked in the browser, not on hardware
3.7	A route for each job category	**All six built** (Workshop, Call out, Walk in, Remote support, General, Recurring). **Job types** (27 October): each company switches off the categories and steps it does not use, without breaking a route — built in the portal and the console, not on hardware (checks 897–911)
3.8	Office chat, browser support, website chat, tickets by email and a contact form, a knowledge base	**Built**, not on hardware. A website chat can be **handed over** (15 October). Tickets by email not yet tried against a real mailbox; Microsoft 365 sign-in for it is later (Clint: "leave it for now")
3.9	Outside help	**All four steps built**, and **a technician asking for it** from a session (22 October). Not on hardware
Free	**The free version**	**All built 6 October**, none on hardware. Not public: signup is off

Stage	What it is	State
4	Hosting, security, seats, payment	**Mostly done.** Live server, TLS, two-factor, invitations, backups (and a nightly copy to the office, built — check it is switched on), end-to-end encryption, the reinstall check: done or built. **PostgreSQL on live since 9 October**; load test run 12–13 October (18 companies per server; the port ceiling found there is fixed and on live). **PayFast (4.8) built in five stages** (13–14 October) and proved against PayFast's sandbox; the live server is still on the sandbox account and the pages are hidden. **Platform admins and the control dashboard** (13 October) built. **Updating the programs themselves (4.6) BUILT 16 October** — customer computers and the console, in rings, with rollback; not on hardware (checks 723–739, 737 above all) — **on hardware 19 October the background services updated themselves; the tray app's fault found there is fixed** (§152). **A company's data export (4.9)** built 22 October. **Monitoring**: a check from the office PC, built 27 October, not installed yet. Not done: automatic placement of companies on servers (control stages C–E)
5	Code signing	**Done.** Sectigo OV certificate issued 14 October 2026 (valid to 18 November 2027), on a YubiKey. **The first signed deploy went live on 15 October** — all 12 programs and scripts the server hands out are signed and timestamped, and `Publish-Platform.ps1` refuses to upload anything unsigned
6	Mac, Linux, Android, iPhone	**Not started.** Deliberately last. (.NET 10, since 14 October, is what a Mac version would need)

How to tell what has been tested

- `docs/TEST-3.6.md` is the numbered checklist of everything not yet proven on real computers — 933 checks, in lettered sections that match the handover sections. Ticked items have been done.
- Every section of `HANDOVER.md` ends with **"Not on hardware"** or says what was proven where.
- `CLAUDE.md` gives a one-paragraph status for every piece of work.
- `tools/testchecklist/` is the internal-testing program on the office share (testers tick checks and log faults; the day's report is folded back into `TEST-3.6.md`).

The pilot companies — next

Clint agreed a deal with **two pilot IT companies**: five seats free while they are pilots (more on request since 2026-10-25, invoiced). On 22 October, a week in with nothing used, he asked for a way to end a pilot — Owner → Pilots → *End the pilot…*, by notice (§153). Each was sent a one-use link made at **Owner** → **Pilots** (or

`pilot-link "<company>" [email]` on the server); it opens `/pilot/{token}`, which works even with signups off and behind the holding page. The pilot guide that goes with it — setting up step by step, with screenshots — is `docs/pilot-brochure/The Remote Support Program - Pilot Guide.pdf` (made from `brochure.html` in the same folder). **Before the pilot links are sent:** confirm the deploy containing `PilotCompanies` (§146) is on the live server (Owner → Pilots exists), and tick **Supervisor** beside any Anywhere I.T login that shows only Owner and works tickets.

Before it is sold to the public

In the order agreed:

1. Finish internal testing (`TEST-3.6.md`) and fix what it finds. Every hardware run so far has found at least one fault that no automatic test could.
2. The pilots' experience, and their faults fixed.
3. The remaining security items in document 03, section 9: an outside penetration test, and making sure the office copy of the backups is running.
4. **PayFast on the live account**: `deploy\Set-PayfastKeys.ps1` with the live merchant details, then `Payfast:Sandbox` false (`docs/NEXT-SESSION-PAYFAST.md`). PayFast's Recurring Billing page already points at the live server.
5. **Publishing the real website** (`Publish-Platform.ps1 -RealSite`) and switching public signup on — Clint's decision, not a developer's. **Decided: 1 November 2026.** Since 27 October `-RealSite` opens signups and our chat bubble too, and **every deploy after that day needs it**. Clint has not yet checked the website's words (`docs/WEBSITE.md` ; `TEST-3.6.md` 924).
6. **Install the site check on the office PC** (`deploy\monitor\Install-SiteWatch.ps1` , document 04), so somebody hears when the site is down before a customer does.

Before the free version is offered to the public: its hardware checks (`TEST-3.6.md` sections BR–BX, checks 459–496 — above all 469, a real free install, and 475, the *Being controlled remotely* bar on a real screen).

Known faults still open

From `docs/BUGS.md` (which records what was tried, so read it before investigating):

#	Fault	State
2	The customer's tray app may keep saying "support in progress" after a session ends	Cause found and fixed in September; left open until seen fixed on a real machine
7	The customer's desktop shortcut logo has a black square behind it	The logo file has no transparent background. Fixed by supplying a transparent PNG logo, not by code

#	Fault	State
8	A file transfer over a really poor line	Fixed and proven on a simulated bad line; never pushed over a real bad one
9	ESET antivirus briefly blocks the health check install	Automatic retries built (1, 5, 15 minutes); not seen on a real machine yet. Signed programs (since 15 October) should also help
10	The **first** copy after the customer logs in is not passed to the technician	**Fixed 6 October** (`HANDOVER.md` §93), not yet seen fixed on a real machine
11	Thousands of machines connecting at once stalled the server	**Found and fixed** during the PostgreSQL work (§121) — it was never the database
12	Past ~28,000 connections the whole server stopped: Caddy ran out of ports to the platform	**Fixed** 13 October: Caddy reaches the platform through a Unix socket; **on live since 15 October**. Memory is now the limit (18 companies per server)
13	A message a machine sends in the same instant as its hello can be lost	Found by a test tool on 18 October. Low: no real program is known to do it. **Not fixed** — `BUGS.md` says how
—	A "ding" sound when pasting on the customer's machine	Never explained. Harmless

Smaller known gaps listed in the handover:

- Photos added later to a multi-machine job always go to machine 1.
- A machine added to a job by mistake cannot be removed (abandon and rebook).
- A workshop device password cannot be changed after the ticket is created.
- The customer is not told when someone types a wrong unattended password.
- Scripts run as the system account, not as the logged-in person.
- *Being controlled remotely* does not show on Windows' secure screens (a UAC prompt, Ctrl+Alt+Del).
- PayFast's sandbox never runs a subscription's scheduled monthly charge, so a renewal has only been proved by the test tool and by a charge pressed by hand; the first real one will be on the live account.
- A company with only owner-only logins and technicians has nobody who may close a ticket (a technician never closes). That is the deal: the owner ticks Supervisor or Office administrator, and takes a seat.
- The SQLite fallback on the live server was meant to be retired about a week after the PostgreSQL switch (§125). Not done yet.

Assumptions Clint may still change (each is a small change): free-version signups wait for approval like any other; an upgraded company starts a 14-day trial; a pilot company is Active at once with no trial.

Dates and renewals to watch

What	When	What happens if missed
Domain `theremotesupportprogram.com` (Afrihost)	Yearly — check the date in Afrihost	**Every installed customer app stops working**
xneelo Cloud server	Monthly billing	The server can be deleted. In September 2026 an earlier server was lost when its xneelo account was disabled, and the supplier kept no copies
xneelo shared hosting (email)	Monthly billing (R99)	Email for the domain stops
Code-signing certificate (Sectigo)	**The first certificate expires 18 November 2027.** The two-year term includes a second one at no charge; Sectigo may check the company again before issuing it	New builds cannot be signed, and `Publish-Platform.ps1` will refuse to deploy. Already-signed files keep working (they are timestamped)
Let's Encrypt website certificate	Every 90 days	**Automatic** — Caddy renews it. Only a problem if the server is broken
Pilot links	30 days after each is made	The link stops working; make another at Owner → Pilots
Microsoft app secret (Tickets by email, *Sign in with Microsoft*), once registered	When it was made to run out — diary it (`docs/MICROSOFT-365.md`)	Every company's Microsoft 365 mailbox stops being read; make a new secret and run `deploy\Set-MicrosoftApp.ps1`
.NET 10	Microsoft supports it until November 2028	Security updates stop; rebuild on the next long-term version before then

What a new team should do first

1. Get the keys and accounts from document 08. Without the SSH key, the YubiKey and the password manager, the live server cannot be maintained or a release signed.

2. Restore the code on a Windows PC, build it, and run the platform locally (document 05).
3. Run a handful of the test tools (`securitycheck` , `workflowcheck` , `unattendedwire` , `pgcheck`) to see them pass — that confirms the build environment is right.
4. Read `HANDOVER.md` §1–14 (the foundations), §112 (end-to-end encryption, before touching anything a session sends), §125 (the live server on PostgreSQL), §147–148 and §152 (automatic updates, before releasing anything), §158–159 (the security review and the addresses), §166–168 (job types, before touching a ticket route), §169 (going public) and §132–146 (the other recent work).
5. Make a test company on a local platform with `tools/workflowdemo` and click around.
6. Only then touch the live server — and take a backup first.

08 — Keys, passwords and accounts

None of the secrets below are on this USB stick. That is deliberate. Anyone holding the secrets could reach every customer computer that allows unattended access, read stored passwords, or take the website down. A USB stick can be copied in seconds without anyone noticing.

This page lists **every** secret and account a new team will need, **where it lives today**, and what to do if it is lost. The right-hand column is for Clint to fill in **by hand, on the printed copy** (or in the password manager), saying where each one is kept. **Do not type passwords into this document.**

How to hand the secrets over safely

Pick one, and write which on the printed copy:

1. **Password manager emergency access (recommended).** Most password managers (Bitwarden, 1Password, LastPass, Keeper) let you name a trusted person who can request access to your vault; if you do not refuse within a waiting period you choose, they get it. Put every item below into the vault, and name the trusted person.
2. **A sealed envelope in the same safe**, holding the password manager's master password and its recovery code (or a printed emergency kit) — not the individual passwords.
3. **An encrypted file on a second USB stick** (for example a 7-Zip archive with AES-256 and a long passphrase), with the passphrase kept somewhere else (the sealed envelope, or a lawyer).

The two most important items are **the SSH key** (row 1) and **the backup passphrase** (row 9). With those and this USB stick, a development team can rebuild everything.

The list

Getting into the live server

#	Secret	Where it is today	If lost	Kept in (fill in)
1	**SSH private key** for `ubuntu@154.65.110.136`, and its passphrase	`C:\Users\Clint\.ssh\id_ed25519` on Clint's laptop; should also be in the password manager	Log in to the xneelo Cloud panel, open the instance's **Console**, and add a new public key to `/home/ubuntu/.ssh/authorized_keys`	

#	Secret	Where it is today	If lost	Kept in (fill in)
2	**xneelo Cloud** panel login (the server)	Clint	xneelo support, with company proof of ownership	

Running the platform

#	Secret	Where it is today	If lost	Kept in (fill in)
3	**Owner key** — signs in to `/owner`	On the server in `/etc/rsp-owner-key`; should be copied to the password manager	Read it again from the server with SSH: `sudo cat /etc/rsp-owner-key`. If it has been exposed, replace it: `deploy\Rotate-Keys.ps1 -Server <ip>` (also replaces the server key)	
3b	**Server key** — lets the control dashboard on another server read and change companies on this one (V2.2)	On the server, in the service settings `go-live.sh` writes (`RSP_Platform__ServerKey`). The owner and server keys were **replaced on 15 October 2026** after a log holding both was pasted into a chat	`deploy\Rotate-Keys.ps1 -Server <ip>` makes new ones; any other server that reads this one is then told the new key at Owner → Servers	
3c	**Platform admins' logins** — each admin's own email, password and authenticator for `/owner` (V2.2)	Each admin	An Owner-level admin, or the owner key with its code, can set the person up again from Owner → Admins	
4	**Owner authenticator** — the second factor for `/owner`	Authenticator app on Clint's phone	On the server: `cd /opt/rsp && sudo -u rsp ./RemoteSupport.Platform reset-owner-2fa`, then sign	

#	Secret	Where it is today	If lost	Kept in (fill in)
			in with the owner key and scan the new QR code	
5	**Clint's company login** `clint@anywhereit.co.za`, its authenticator, and its **ten recovery codes**	Clint; authenticator on his phone	Another owner or supervisor can Reset two-factor or send a reset link from the People page; or on the server: `cd /opt/rsp && sudo -u rsp ./RemoteSupport.Platform reset-2fa anywhereit clint@anywhereit.co.za`	
6	**Platform mailbox password** (the mailbox the platform sends from, set in Owner → Mail)	Stored encrypted on the server; the mailbox itself is at xneelo shared hosting	Reset the mailbox password in konsoleH, then enter it again in Owner → Mail	
6b	**Forwarding mailbox password** (`tickets@theremotesupportprogram.com`, for tickets by email — Owner → Mail, *Forwarding mailbox*; only once it is set up)	Stored encrypted on the server; the mailbox is at xneelo shared hosting	Reset it in konsoleH, then *Save and test* it again in Owner → Mail	
6c	**Each IT company's support mailbox password**, when they let the platform check their mailbox (Portal → Tickets by email)	Stored encrypted on the server, one per company. Theirs, not ours	The company types it again on that page	

Backups and stored data

#	Secret	Where it is today	If lost	Kept in (fill in)
7	**The data folder's encryption keys** (`/opt/rsp/data/keys`)	On the server, and inside every nightly backup	Every stored workshop device password and mailbox password becomes unreadable. Everything else survives	(inside the backups)
8	**Recent backup files** (`/var/backups/rsp/*.tar.gz.gpg`)	On the server (the `rsp-files` volume). Since 8 October an **office PC can fetch each night's copy** onto the office share (`deploy/backup/README.md`) — **check it has been set up and is running**; until it is, they are only on the server	If the server is lost too, and there is no office copy, all company data is lost	
8b	**The office PC's key for fetching backups** (the `rspcopy` account; it can only list and fetch the encrypted files)	On the office PC that runs `Pull-Backup.ps1`	Run `Install-BackupPull.ps1` again on an office PC (`deploy/backup/README.md`)	
8c	**The site check's mailbox password** (V2.4) — the theremotesupportprogram.com mailbox the office PC sends *site is down* emails through	On the office PC that runs the site check, in `%USERPROFILE%\.rsp-watch\mail.xml` , readable only by that Windows account on that PC	Run `Install-SiteWatch.ps1` again on the office PC and type it (`deploy/monitor/README.md`)	
9	**Backup passphrase** (`/etc/rsp-backup-key`)	On the server; should be copied to the	The backups cannot be opened.	

#	Secret	Where it is today	If lost	Kept in (fill in)
		password manager	Make a new one by deleting the file and re-running `go-live.sh`; older backups stay locked for ever	

> **The database has no password.** PostgreSQL on the live server (since 9 October 2026) is reached only over a local socket by the platform's own Linux account; nothing to keep.

The domain, DNS and email

#	Account	Used for	If lost	Kept in (fill in)
10	**Afrihost** login	Domain registration and renewal	Afrihost support, with ID and company documents	
11	**Cloudflare** login	DNS — where the name points	Cloudflare account recovery	
12	**xneelo konsoleH** login	Shared hosting and the domain's mailboxes	xneelo support	
13	**Mailbox passwords** (`info@`, `queries@` and others on the domain)	Company email	Reset in konsoleH	

Code signing

#	Item	Where it is today	If lost	Kept in (fill in)
14	**The YubiKey** — a YubiKey 5 NFC FIPS, serial **38992498**, firmware 5.7.4. The code-signing private key is inside it, in **PIV slot 9a**	In the safe. Plugged in only to sign a release	**A new certificate must be bought and validated again** — roughly R10,000 and some	

#	Item	Where it is today	If lost	Kept in (fill in)
			weeks. There is no other copy; see the note below	
15	**Its PIN**, and the **PUK** that unblocks the PIN	On paper, kept **apart from the token**	Three wrong PINs block it and the PUK is the only way back. Lose both and the key is gone	
16	**LeaderSSL** account (reseller) and **Sectigo** order details — proforma 2611897, reference 80819755/2611897	`info@theremotesupportprogram.com`	LeaderSSL support, `info@leaderssl.com`	

The code-signing key cannot be backed up, and that is deliberate. The private key was generated *inside* the YubiKey on 8 October 2026 and cannot be read out of it by anybody, including us — which is exactly what the certificate authority requires and what the attestation certifies. The consequence is that **the token is a single point of failure**: lose it, destroy it, or forget the PIN, and the certificate is worthless and must be bought again from the beginning.

The management key is stored on the token itself, protected by the PIN, so **the PIN is the only secret a person has to keep**.

Never run either of these against this token:

```
ykman piv reset            # wipes everything
ykman piv keys generate    # quietly replaces the key in the slot
```

Either destroys the key the certificate was issued against. The certificate only works with that one private key.

`docs/CODE-SIGNING-ORDER.md` is the full story of how it was bought and set up, including the steps to repeat at renewal.

The code itself

#	Item	Where it is today	Note	Kept in (fill in)
17	**The source code and its history**	Clint's laptop, `C:\Users\Clint\Desktop\Remote`	**There is no online copy** (no GitHub or similar). If the laptop and this stick are both lost, the code is gone. Consider a	This USB stick

#	Item	Where it is today	Note	Kept in (fill in)
		`Support`, **and this USB stick**	private online repository as a second copy	
18	Test-tool login file `tools\rs-credentials.json`	Clint's laptop only	Only for running test tools against the live server. Recreate it by hand (format in `tools/Common/RsCredentials.cs`)	not needed
19	`branding\brand.json` with the tenant key	Clint's laptop only	**Not a secret.** Download it again from the portal (Downloads page)	not needed
20	The office test platform's database	Clint's laptop, `src\RemoteSupport.Platform\bin\Release\net10.0\data`	Test data only	not needed

Company and business documents

#	Item	Where it is today	Kept in (fill in)
21	CIPC registration — The Remote Support Program (Pty) Ltd, 2026/737434/07	Company records	
22	The investment agreement and other legal papers (`legal\` folder on the laptop, deliberately not in the code repository)	Clint's laptop	
23	**PayFast** merchant account login (the website), and the account's **merchant ID, merchant key and passphrase** — one set for the **sandbox** (testing) and one for the **live** account (V2.2)	The login: Clint. The ID, key and passphrase: on the live server (stored encrypted), put there with `deploy\Set-PayfastKeys.ps1` — never typed into chat. The sandbox set used for testing is on Clint's laptop, encrypted to his Windows	Read them again in PayFast's dashboard (Settings) and run `Set-PayfastKeys.ps1` again (`-Live` for the live account). **If the server's address changes, change the allowed address on**

#	Item	Where it is today	Kept in (fill in)	
		user (`tools\payfast-sandbox.xml` , made by `tools\Save-PayfastSandbox.ps1`)	**PayFast's Recurring Billing page too**	
23b	`tools\rs-postgres.txt` — how the test tools reach a local PostgreSQL	Clint's laptop only	Test only; recreate for a local PostgreSQL (format in `tools/Common/HarnessDb*.cs`)	not needed
23c	The **Microsoft Entra app** *The Remote Support Program* (Tickets by email's *Sign in with Microsoft*, HANDOVER §151): its **Application (client) ID** and **client secret** (it runs out — two years at most), and the **Microsoft Cloud Partner Program** account used for publisher verification	The app: App registrations in the Microsoft Entra tenant it was registered in (Clint's admin login). The ID and secret: on the live server (secret stored encrypted), put there with `deploy\Set-MicrosoftApp.ps1` — never typed into chat; `microsoft-app show` on the server says when the secret runs out. The Partner Center login: Clint	Make a new secret in Entra (App registrations → Certificates & secrets) and run `Set-MicrosoftApp.ps1` again **before** the old one runs out — every company's Microsoft sign-in stops the day it does. All the steps: `docs/MICROSOFT-365.md`	

Test machines (Anywhere IT office only)

#	Item	Note
24	Windows logins for DESKTOP-OPHQIDH and DESKTOP-2N6FVGT	Only needed to keep testing
25	The office shared drive `\\192.168.1.33\anywhereit` (WD My Cloud)	Holds the Test Checklist results

---

Keys that stay on each computer (V2.1 — never collected, never backed up)

End-to-end encryption (document 03) uses keys that **must not** be gathered anywhere central — that is the point of it. Nobody needs to hand these over.

Item	Where it is	If lost
Each customer computer's **identity key**	`%ProgramData%\RemoteSupport\identity.key` on that computer, readable only by Windows' system account	A new one is made. Every console that knew the old one stops at the next connection and asks the technician whether to trust the new one
Each console's **remembered identities** ("pins")	`%LOCALAPPDATA%\RemoteSupport\machine-keys.json` for each technician on each PC	The console simply learns each computer again at its next connection

---

Things that are NOT secrets (so nobody wastes time protecting them)

- The **tenant key** in each company's download. It is public by design (document 03, section 5).
- The live server's **IP address** and the **domain name**.
- The code itself contains no passwords. Test tools use made-up passwords on throwaway test servers.

09 — Glossary

Words used in this pack and in the technical documents, in plain English.

Word	Meaning
Agent	The background Windows service on a customer's computer that makes unattended access possible
Attended	A session where the customer is at the computer and clicks Allow
Authenticator app	A phone app (Google or Microsoft Authenticator, etc.) that shows a new six-digit code every 30 seconds, used as the second step of signing in
Caddy	The web server program on the live server that handles the security certificate and passes traffic to the platform
Client	(1) A customer of an IT company, in the ticketing system. (2) The customer's tray app program, `RemoteSupport.Client`. The meaning depends on context
Code signing	A digital signature on a program proving who made it and that it has not been altered. Without one, Windows warns about or blocks the program. Every program the live server hands out has been signed since 15 October 2026
Control dashboard	The `/owner` page showing every server, how full it is, and every company on it (V2.2)
Damage report	A workshop step for a machine not worth repairing: a report (often for insurance) attached and emailed to the client (V2.2)
Consent	The customer's permission, given by clicking Allow on the prompt
Console	The technician's Windows program
Data Protection keys	The server's encryption keys (in `data/keys`) used to lock stored passwords
End-to-end encryption	The session sealed between the customer's computer and the technician's console with a key the server never sees, so the server passes it on without being able to read it (V2.1)
DNS	The internet's address book: it turns `theremotesupportprogram.com` into the server's number. Ours is managed at Cloudflare

Word	Meaning
Free version	One person reaching two of their own computers, unattended, each with a password; no tickets or team. Upgradeable in place to the full version (V2)
Full version	Everything else in this pack: an IT company supporting its customers, with a team, tickets and branding
Harness / test tool	A small program in `tools\` that automatically checks one feature and reports how many checks passed
Identity key	A key made on each customer computer that proves, in every session, that it is the same computer. Each console remembers it (a **pin**) and asks before going on if it ever changes (V2.1)
Holding page	The single public web page shown at the website's home address until the real site is published; it carries the terms, refund and privacy policies
Hash	A one-way scrambled version of something (like a password). You can check a password against it, but cannot turn it back into the password
Host	The program started on the customer's computer for each session; it captures the screen and carries out the technician's actions
H.264	The video compression standard (the same as most online video) used to send the screen picture efficiently
Hub	The part of the server that keeps track of who is online and passes messages between programs; one per IT company
Migration	A recorded change to the database's structure, applied automatically when the server starts
Release	One build of the programs, as the server hands it out for computers to update to; it carries a version (V2.3)
Ring	Who a release is offered to so far: our own company first, then the pilots, then everybody. Moved on, or paused, on Owner → Updates (V2.3)
Rollback	A computer putting its old programs back because a new version did not connect within five minutes. It pauses the release for everybody (V2.3)
Update by hand	The one-off update a computer installed before 16 October 2026 needs (Tools → Scripts → *Update this machine*) before it can update itself (V2.3)

Word	Meaning
Version	The moment a program was built, in UTC: 2026.10.16.1432 is 14:32 on 16 October 2026. "1.0 (old)" is a computer from before version numbers (V2.3)
Owner (platform)	The Remote Support Program itself — signs in to `/owner`
Owner (tenant)	The owner of one IT company, e.g. the owner of Anywhere IT. An **owner-only login** (Owner ticked and nothing else) is free, oversees and comments, and does not work tickets (V2.2)
PayFast	The South African payment company that holds companies' cards and charges them monthly. We never see a card number (V2.2)
Pilot company	One of the first IT companies, signed up from a one-use link: five seats free while a pilot (V2.2); ended by notice from Owner → Pilots (2026-10-22)
Platform admin	A named person who may sign in to `/owner`, at one of four levels, with everything they do logged (V2.2)
PostgreSQL	The database the live server has used since 9 October 2026 (before that, SQLite, which tests still use)
Payload	The set of program files offered for download, with a company's branding added
Plan	Whether a company is on the free or the full version — a setting on the company, enforced by the server
Portal	The website staff of an IT company sign in to: tickets, clients, settings
Job types	The portal page where a company switches off the ticket categories and steps it does not use. Each ticket keeps the switches from the day it was opened
Recurring task	A schedule (daily, weekly or monthly) that opens an ordinary ticket on each day it is due — the sixth ticket category (V2.1)
Recovery codes	Ten one-use codes given when two-factor is set up, for signing in without the phone
Relay	Passing data through the server from one end to the other without storing it
Remembered (saved) password	A machine's unattended password that a technician has typed correctly once, so they are not asked again. The server keeps only that it was typed right, never the password

Word	Meaning
Removed machine	A machine taken off a company's list by a supervisor or the owner, with a reason, or uninstalled by the person at it. Its history stays; it is refused if it ever comes back
Route / workflow	The set of stages a ticket of a particular category moves through. Every step can be undone with **Back** (V2.2)
Sandbox	PayFast's test system: payments that move no real money. The live server is still on it until go-live
Server key	A secret one server uses to read and change companies on another, for the control dashboard (V2.2)
Seat	A paid place for one person in a company's subscription — any role; only an owner with no other box is free
Security stamp	A hidden value on each person that changes when their password or permissions change; changing it signs them out everywhere at once
Session	One live remote-control connection between a technician (or several) and a customer computer
Ship	Building and putting a new version where it will be used (`ship-build.ps1` for the office test, `Publish-Platform.ps1` for live)
Slug	A company's short web name, e.g. `anywhereit` in `/get/anywhereit`
SSH	Secure remote login to the Linux server, using a key file instead of a password
SYSTEM	Windows' most powerful built-in account. The host and agent run as it so they can work on secure screens and administrator programs
Tenant	One IT company using the platform (Anywhere IT is a tenant)
Tenant key	The long code in a company's download that tells the server which company a customer computer belongs to. Public, not a password
TLS	The encryption that protects web traffic (the padlock in the browser)
Tray app	The customer's small program by the clock
Two-factor (2FA)	Signing in with something you know (password) and something you have (phone with an authenticator app)

Word	Meaning
UAC prompt	Windows' "Do you want to allow this app to make changes?" prompt. It appears on a special secure screen, which ordinary remote tools cannot see
Unattended	A session started by a technician without anyone at the customer's computer, allowed only if the customer has ticked the box
Unattended password	An optional password the customer sets on their own machine; needed for any unattended connection to it. Required on every free-version computer, chosen when it is installed
Upgrade	Turning a free-version account into a full one, in place: same account, computers and saved passwords
V1, V2, V2.1, V2.2	Versions of this handover pack: V1 written 2 October 2026, V2 on 6 October 2026, V2.1 on 7 October 2026, V2.2 on 15 October 2026

10 — What is new in V2

V1 of this pack was written on **2 October 2026**. **V2** was written on **6 October 2026**. In those four days a great deal was added. This page lists all of it, in the order it was built, so somebody who read V1 can see what changed without reading everything again. Documents 01–09 have been updated to match. **What came after V2, on 7 October, is document 11 (V2.1).**

Every item below was **built and passed its automatic test tools**, but **none of it has yet been tried on real customer computers** unless it says so. The hardware checklist is `docs/TEST-3.6.md`, and the section of `docs/HANDOVER.md` named in brackets is the full engineering record.

The biggest change: a free version

A second way to use the product, for **people with remote offices** — one person reaching their own computers in their own branch offices. Clint's decisions, in full, are in `docs/NEXT-SESSION-FREE.md`. Document 02, Part H describes it.

What	In short	Record
One person, two computers	The owner is also the technician. At most two computers, counted by the actual machine (a reinstall is not a third). A computer can be removed to free its place, twice a calendar month	§101
Unattended only, with a password	Nobody at a free computer is ever asked anything. Every free computer must have an unattended password, chosen when it is installed	§101, §103
What it can do	Remote control, Ctrl+Alt+Del and restart, files, copy and paste, printing both ways. No tickets, chat, scripts, sound, recording or browser support	§101
Signing up	The same signup page: country, then **Free or Full**, then the terms. The free version has its own short section of terms ahead of the region's terms. No logo or colours asked; company name optional	§102
Installing a computer	The download asks the server first whether this computer may be added (so a third is turned away before anything is installed), installs only the background service, and sets the password. **No tray icon stays on a free computer**	§103
"Being controlled remotely"	Because there is no tray app, a small amber bar at the top of every monitor names who is connected, for the whole session. The technician's picture does not show it	§104

What	In short	Record
Its portal and console	Portal: Your computers, Downloads, My account, Upgrade — every other page is refused. Console: just the computers, Connect (password only, asked once), Remove	§105
Upgrading	One page: the full terms and the price (free → R500 a month after a 14-day trial). The same account, computers and saved passwords carry on. An open console switches to the full one at once	§107

The free version is not public yet. Signups are still switched off on the live server, and the public front of the website is still the holding page.

Changes to the full version

Customers' machines and sessions

What	In short	Record
Remembered unattended passwords	Once a technician types a machine's unattended password correctly, they are not asked again — from any computer they sign in on. A colleague who never typed it still is. Changing the password at the machine forgets it for everyone. A supervisor or the owner can see who has it remembered (names only) and clear it	§100
Removing a machine, with a reason	Supervisors and the owner, from the console (right-click) or the portal (Devices). Reasons: Upgraded, Replaced, Private device, No longer a customer, Other (with a note). Its tickets and history stay; a **Removed machines** list shows who, when and why. An **online** machine uninstalls itself; an offline one is refused when it comes back	§106
Uninstall in the tray app	For a customer who no longer wants it. Windows asks for administrator permission; the machine then leaves the IT company's list as "Uninstalled at the machine"	§106
Remote printing, both ways	A technician can print what the customer prints on their own printer, and print a file on the customer's printer	§95
Wake-on-LAN	Right-click an offline machine → Wake up. Another of that customer's machines on the same network sends the wake-up — nothing to set up on the customer's router	§94

What	In short	Record
Outside help	A customer can let somebody outside their IT company in with a 9-digit code: another company on the platform, a guest with the guest console, or a guest in a browser. Screen and control only; files only if the customer ticks it	§87, §89–§92
Processes and Performance	Right-click a machine to see its processes and live graphs without connecting	§81

Faults fixed

Fault	Fix	Record
BUGS #10: the first copy after the customer logged in did not reach the technician	Fixed	§93
On some computers a program opened or closed did not appear until Refresh	The graphics chip's colour converter was only asked once per picture; it is now asked until it has finished	§99
A reinstalled machine left an offline "twin" in the machine list until the server restarted	The old entry is removed when the reinstall is recognised	§101
The console and customer app had square boxes, black corners and a mixed colour rule	One look: everything rounded, one colour rule. A test tool now checks the corners of 28 windows in both themes	§98

The office side

What	In short	Record
Tickets by email and a contact form	An email to the support address, or the website's contact form, opens a ticket and replies with its number	§79
Knowledge base	General and per-client articles, with folders, history, and link buttons	§80, §96
One sign-in at a time	One console and one browser per person; a second computer signs the first out	§82

What	In short	Record
Seats are a price the owner agrees to	Adding a person past the package shows the new monthly price, and only the owner can accept	§83
Signup by country, with that country's terms	Five regions, each with its price and its server; a waiting list where there is no server yet	§84
The company's own time zone	Every time on the portal is in it	§85
Call-outs can be rescheduled; billing notices	The office is told when a job reaches billing	§86
Quotes	Several photos per item; the request asks stock, ETA and delivery fee; signed with a chosen signature	§96, §97
Terms are no longer marked DRAFT	Clint, 6 October: "assume they are correct as is". His lawyer's changes, when they come, are edits and a new version number. The regions not yet open (US, Canada, UK, Europe) still have a few facts to fill in before they open	§88, §102

New test tools since V1

`freecheck`, `freeinstall`, `freeportal`, `noticeprobe`, `upgradecheck`, `removecheck` (the free version and removing machines); `wakecheck`, `printcheck`, `clipfirst`, `lookprobe`, `outsidercode`, `outsidervisit`, `guestcheck`, `mailintakecheck`, `kbcheck`, `watchcheck`, `onesignin`, `seatcheck`, `signupcheck`, `clockcheck`, `billingwire`; and `unattendedpw` grew from 33 to 65 checks. Each one starts its own throwaway server where it needs one. Run the relevant one before and after any change.

What to do with V2

Nothing here changes the advice in document 07: **the next job is proving, not building.** The hardware checks for everything in this page are `docs/TEST-3.6.md` sections **AV–BX** (checks 320–496).

11 — What is new in V2.1

V2 of this pack was written on **6 October 2026**. **V2.1** was written on **7 October 2026**. This page lists what was added in that one day, so somebody who read V2 does not need to read everything again. Documents 00–09 have been updated to match; document 10 still lists what V2 added to V1.

Every item below was **built and passed its automatic test tools**, but **none of it has yet been tried on real customer computers**. The hardware checklist is `docs/TEST-3.6.md` (the lettered section is named in each row), and the numbered section of `docs/HANDOVER.md` is the full engineering record.

The biggest change: sessions are encrypted end to end

Before this, the connection between each program and the server was encrypted, but **inside the server** a session could in principle be read: the picture, every key typed (passwords included), the clipboard, files, sound and scripts. Now the customer's computer and the technician's console agree a key **the server never sees**, and the server only passes sealed data along.

What	In short	Record
Sealed sessions	Everything a session carries is sealed with AES-256-GCM between the two ends. The server still sees who connected to what and when, and the audit lines a ticket needs (file names moved, a script's first line, restarts)	§112, CC
Each computer has an identity	A key made on the customer's computer, readable only by Windows' system account, signs every session. Each console remembers it; if it ever changes (a reinstall, or somebody pretending) the console **stops and asks** before showing or sending anything	§112, CC
It says which	The session window shows a green *End-to-end encrypted*, or *Not end-to-end encrypted* when an older program is involved — older programs still work, by Clint's decision	§112, CC
The browser too	Browser support and the guest's browser page seal the same way	§112, CC

The honest limit: a server that had been taken over could still *start* a session of its own; it could not read or take over a real technician's. Document 03 explains it.

Other new work

What	In short	Record
Recurring tasks — a sixth ticket category	Set up once — daily, weekly on chosen days, or monthly — for maintenance, billing, readings or the company's own types. A ticket opens for its people at 06:00 company time each day it is due; Done closes it, or To be invoiced. Cancel stops future tickets only	§108, BY
A ding and a ring	A soft ding for every chat message from somebody else; a soft ring every four seconds while a support request or a website visitor waits. A bell in the console mutes them on that PC; the portal has its own button	§109, BZ
Windows shortcuts go to the customer's computer	While the picture has the keyboard, the Windows key and its shortcuts, Alt+Tab, Ctrl+Esc, Ctrl+Shift+Esc and Print Screen act over there, not on the technician's PC. No switch, by Clint's decision. Win+L and Ctrl+Alt+Del cannot be taken by any program — the toolbar has buttons for both	§110, CA
A reinstalled machine waits for the office	A machine recognised after a reinstall is matched only by its hardware, which any program on it can read. So unattended access now waits until a supervisor or the owner confirms it is the same machine (right-click → Confirm, or the Devices page)	§111, CB
Passwords scrambled harder	600,000 rounds instead of 120,000. Each person's is re-made at their next sign-in; nobody is signed out	§111, CB
Branding reaches installed machines	When a company changes its logo, colours, wording or permission-prompt settings, every installed customer computer takes the new look — at once if it is on, otherwise when it next connects. Only what a person sees travels, never where the computer connects. The tray app restarts itself quietly, only when nobody is using it	§113, CD
The console's windows carry the company's logo	Before, only the main window did; the sign-in, session, ticket and chat windows showed Windows' default icon, and so did the console's shortcuts	§114, CE

The last two security items agreed on 1 October — the hardware-fingerprint takeover and the password rounds — are now **built** (document 03, section 9). What remains there is code signing, an outside penetration test and a copy of the backups off the server.

New test tools

`recurringcheck`, `soundcheck`, `shortcutcheck` (it presses real keys — leave the keyboard alone while it runs), `reinstallcheck`, `hashcheck`, `e2ecore`, `e2ewire`, `e2einterop` and `brandpush`. `iconcheck` and `consoleexecheck` gained checks. Each one starts its own throwaway server where it needs one.

Putting V2.1 live

All of it goes out with `deploy\Publish-Platform.ps1` (two database changes, `RecurringTasks` and `ReinstallConfirm`, applied by themselves), then **every technician re-downloads the console**. The encryption, the reinstall check and branding also change the customer's programs, so they need a **full ship and a reinstall** on customer computers. Document 05 has the commands.

Changed on 8 October 2026: five seats, ten at most

Clint changed the licence after V2.1 was first made. The package is now **five seats** (still R500 / $49 / £49 / €49 a month), extra seats are still R300 / $20 / £20 / €20, and **no company can have more than ten seats**: the product is for small and medium IT companies. One owner-only login is free; everybody else takes a seat, a second owner-only login included. The South African terms say so (version 2026-10-08), and so does the holding page. Document 02 (G6) is updated; `docs/PRICING.md` and section 116 of `docs/HANDOVER.md` are the record. It goes live with `deploy\Publish-Platform.ps1` (database change `FiveSeatPackage`, which moves every company to five); nothing on customer computers changes.

The same day, **a company may have several owners** (section 117): only an owner can make somebody an owner, change an owner's roles or remove one; there is always at least one; and every owner with no other role, past the first, takes a seat.

Added on 9 October 2026: updating the client from a ticket

A comment can be emailed to the client from the ticket — the fault the ticket was opened for and that one comment, with the writer's name, from the company's support address; the client's reply comes back onto the same ticket and flags the people on it. Signup asks for the support address. Document 02 (C4b) is updated; section 123 of `docs/HANDOVER.md` is the record. It goes live with `deploy\Publish-Platform.ps1` (database change `ClientUpdates`) and a console re-download; nothing on customer computers changes.

12 — What is new in V2.2

V2.1 of this pack was written on **7 October 2026**. **V2.2** was written on **15 October 2026**. This page lists what was added in those eight days, so somebody who read V2.1 does not need to read everything again. Documents 00–09 have been updated to match; documents 10 and 11 still list what V2 and V2.1 added. (Three items from 8 and 9 October — five seats, several owners, emailing the client from a ticket — were already added to the end of document 11; they are listed here too.)

Unless a row says otherwise, everything below was **built and passed its automatic test tools**, but has **not yet been tried on real customer computers**. The hardware checklist is `docs/TEST-3.6.md` (the lettered section is named in each row), and the numbered section of `docs/HANDOVER.md` is the full engineering record.

The biggest changes

What	In short	Record
The live server runs on PostgreSQL	Since 9 October 2026, 11:56 UTC. Rehearsed first on a copy of live (all twelve steps), then switched: down 11 seconds, all 2,538 rows identical. The database and the data each have a disk of their own. Every test tool passes on both databases; **every model change now needs two migrations** (document 05). The way back to SQLite exists and is tested	§118–§121, §125
Load-tested on xneelo	18 companies per server (each 10 technicians, 500 machines with agents, 10 sessions) — about 18,500 connections on a 4-processor, 8 GB server. The first ceiling found (Caddy running out of ports) was fixed — Caddy now reaches the platform through a socket, **on live since 15 October** — so memory is the limit now	§126, BUGS #12
Code signing is in use	The Sectigo certificate was issued on 14 October; the **first signed deploy went live on 15 October**: all 12 programs and scripts the server hands out are signed and timestamped, and `Publish-Platform.ps1` refuses to upload anything unsigned	`CODE-SIGNING.md`
Card payments through PayFast	A full-version signup in South Africa ends at PayFast (R0 now, the package after 14 days); PayFast's notice switches the company on. The amount follows the number of people; receipts; reminders and suspension when a payment does not come (14 days' grace); cancelling. Proved against PayFast's sandbox; **the live server is still on the sandbox and the pages are hidden**	§134–§137, CQ–CT

What	In short	Record
Platform admins and a control dashboard	Named people for `/owner` at four levels, with a log; a card per server showing how full it is, emailed when it crosses 70% or 85%; companies on other servers changed through their own server. Suspending a company now takes effect at once	§132–§133, CO–CP
Pilot companies	Five seats free for as long as they are customers, extra seats invoiced; a one-use link opens `/pilot/…`, Active at once with no trial and no card. The guide that goes with it: `docs/pilot-brochure/The Remote Support Program - Pilot Guide.pdf`	§146, DC
The owner-only login oversees	The free owner login runs the company and comments, but no longer works tickets (it could, through the portal, before)	§146, DC
Everything on .NET 10	.NET 8 leaves Microsoft's support on 10 November 2026. **Proven on hardware**: every test tool, and the new programs on Windows 10 (checks 678–683)	§141, CX

Tickets

What	In short	Record
Damage reports	*Not repairable → Damage report needed*: a note, the report attached, time and amount; emailed to the client (never the time or amount)	§128, CL
Files and photos on any comment	Uploaded as picked; a comment can be files alone; they go with the comment if it is emailed	§128, CL
Go back a step, at every stage	With a reason; the undone step stays in the story crossed through; warns about invoices, payments and emails	§128, §131, CL
Comment boxes grow	Up to 15 lines, in the portal and the console	§128, CL
Edit a ticket	Subject, description, appointment, category (a new category restarts its route)	§130, CN
Sort tickets by date	Portal and console; the dashboard's Unassigned count fixed	§130, CN

What	In short	Record
Job types in colour; the portal menu as tabs		§124, CJ
Linking a ticket to a bigger job no longer closes it	The job waits for its parts, and says which	§122, CH
Part of / not part of a job; unlink a machine from its customer		§122, CH
Emailing the client from a ticket (9 October)	The original fault and that one comment, from the company's support address; their reply comes back onto the ticket	§123, CI

Remote control and the customer's side

What	In short	Record
A picture that never stays blurry	Key frames a minute apart; a still screen sharpened for two seconds after a change	§142, CY
Two sessions at once	One per monitor, two at most	§143, CZ
Copy on one machine, paste on the other — straight between them	Files go from machine A to machine B through the server, sealed, never through the technician's PC	§144, DA
Off means offline	The server notices a machine that has gone silent within about 45 seconds	§122, CH
Reconnecting asks nothing	Within 30 minutes of the end of the last session, the same ticket carries on	§122, CH
Asking for help: subject, description, a picture of the screen	And the app remembers the customer's name, and says when the picture shows only the desktop	§122, §129, CH, CM
A new machine's two hellos at once	A first install's tray app and agent arriving together no longer lose one's details	§127

Chat, the company and the website

What	In short	Record
Hand over a website chat	To anybody signed in now; the visitor is told; the dings follow it	§145, DB
Reviews from companies	Asked once a week in; published only after an admin approves and the company ticks yes	§139, CV
The real website	About a dozen pages, light and dark, screenshots of the real windows with made-up companies, a cookie line. **Hidden behind the holding page** until Clint says so	§138, §140, CU, CW
Five seats, ten at most (8 October)	R500 for five seats in any mix of roles; R300 per extra seat; ten at most; one owner-only login free	§116, CF
Several owners (8 October)	Only an owner makes or removes another; always at least one	§117, CG

Security and running it

What	In short	Record
The owner key opens nothing alone	Two old addresses that took the key without a code were removed	15 October, `securitycheck` 78
Keys are never printed, and were rotated	`go-live.sh` says where a key is, never the key; `deploy\Rotate-Keys.ps1` replaced the owner and server keys on 15 October after a log was pasted into a chat	document 03, 8b
Backups to the office	An office PC fetches each night's encrypted backup through an account that can only list and fetch. Built and tested; **check it is set up**	§120

New test tools

`pgcheck`, `pgcopycheck`, `switchcheck`, `backupcheck` and `Run-DbHarnesses.ps1 -Postgres` (the database move); `loadtest`, `loadstatscheck` and `unixsockcheck` (the load test and the socket); `silentoffline`, `reasonwire`, `requestwire`, `firsthello`; `clientupdatecheck`, `ticketstepcheck`, `ticketeditcheck`; `admincheck`, `controlcheck`; `payfastcheck` (it talks to PayFast's real sandbox) and `signuppaycheck`; `sitecheck`, `siteshots`, `reviewcheck`; `sharpwire`, `keyframewire`, `codecbench --sharp`;

`sessionlimitwire`, `twosessions`, `bridgewire` (two real hosts); `pilotcheck`. Most that make a database run on **both** SQLite and PostgreSQL.

Putting V2.2 live

The platform side goes out with `deploy\Publish-Platform.ps1` (its database changes apply by themselves; it now refuses to upload unless every program is signed, so plug the YubiKey in). Then every technician re-downloads the console. The sharper picture, two sessions and copying between machines, the request window and the remembered name change the customer's programs too, so those need a **reinstall on customer computers**. Before the pilot links go out, check the deploy holding `PilotCompanies` is on the live server (Owner → Pilots) — see document 07.

13 — What is new in V2.3

V2.2 of this pack was written on **15 October 2026**. **V2.3** was written on **16 October 2026**. One thing was added in that day, and it is a big one: **every program now keeps itself up to date.** Documents 00–09 have been updated to match; documents 10–12 still list what V2, V2.1 and V2.2 added.

Everything below was **built and passed its automatic test tools**, but has **not yet been tried on real customer computers**. The hardware checklist is `docs/TEST-3.6.md`, sections **DD** and **DE** (checks 723–739), and `docs/HANDOVER.md` **§147–§148** is the full engineering record. The plan it was built from, with Clint's decisions, is `docs/NEXT-SESSION-AUTOUPDATE.md`.

Why it matters

Before this, a customer's computer ran whatever version was installed on it, for ever. Every fix to the programs on customers' computers reached them only when somebody reinstalled by hand — which, with a few hundred machines per IT company, is not a way to run a business. Updating was held back until code signing worked (15 October 2026), because a program that installs other programs on customers' computers must be able to prove they are ours.

Clint's two decisions: customer computers update **silently** (nobody at the machine is asked), and **the rollout is ours alone** — an IT company cannot hold its machines back, or a security fix would never reach them.

What was added

What	In short	Record
Real version numbers	Every program used to say 1.0.0. Now each build is stamped with the moment it was made, in UTC: **2026.10.16.1432** is 14:32 on 16 October 2026. Shown in the console's machine list and title, on the portal's Devices page (amber when older than what the server hands out), and on the control dashboard. Every computer installed before 16 October shows **1.0 (old)**	§147, DD
Only our signed, newer files are installed	Each computer checks every file itself with Windows' own signature check: signed by **The Remote Support Program (Pty) Ltd** (South Africa), not revoked, and **newer** than what it has — so even somebody who took over our server could not push an old or foreign program. There is no "test mode" that relaxes this	§148 B

What	In short	Record
Releases go out in rings, and can be paused	A new release goes first to **our own company's** computers, then — when a Full admin or Owner says so on **Owner → Updates** — to **the pilots**, then **everybody**. A few machines a minute, so the server is never flooded. **If a computer has to roll back, the release pauses itself** and every Full admin and Owner is emailed	§148 C
Computers update themselves	The background service downloads the release from the IT company's own server, checks every file, waits until **nobody has been connected for 10 minutes**, and installs it. If the new version has not connected back within **5 minutes**, the old one is put back by itself, and that version is never tried on that computer again. The tray app restarts into the new version when nobody is using it; the monthly health check follows by itself	§148 D
The console updates itself	Before its sign-in window, an installed console fetches a newer version and restarts in it ("*Updating the console…*" for a few seconds). While open, an **Update ready** button appears in its header. If a new console cannot reach its sign-in window, the old one is put back	§148 E
The one update by hand	Computers installed before 16 October cannot update themselves yet. **Once each**: connect from the console, then **Tools → Scripts → Update this machine**, and Run. It keeps the computer's identity and settings (a reinstall would make the office confirm it again). The portal's Devices page counts them and marks each one *update by hand once*	§148 F

New test tools

`versioncheck` (version numbers), `updatetrust` (the signature check, with real files — including a certificate made with our name, which is refused), `updatewire` (rings, Pause, the automatic pause, never offering a refused version again — on both databases), `updateapply` (the file swap and the rollback on a computer), `consoleupdate` (two real consoles, one updating the other), `updatebyhand` (the script, run for real against files it must refuse). On 16 October **all 82 database test tools passed**, and the console's window checks (`dashtabs` , `toolbarprobe` , `fitprobe` , `lookprobe`).

Putting V2.3 live

1. **A signed deploy:** `deploy\Publish-Platform.ps1` with the YubiKey plugged in. Two database changes (`AppVersions`, `AutoUpdate`) apply by themselves. **Owner → Updates** then shows the release offered to our own company only.
2. **Every technician re-downloads the console once.** The old consoles cannot update themselves.
3. **Every customer computer gets its one update by hand** (Tools → Scripts → Update this machine).
4. **From the next deploy on**, our own computers update themselves within minutes; watch them come through it on Owner → Updates, **then** offer it to the pilots, then everybody.

The check that matters most is **737 in `TEST-3.6.md`: a deliberate rollback on a real computer**. Until it has been seen working, treat automatic updating as unproven.

Known gaps

- **A restart of the computer during the five minutes after an install** leaves nobody to roll back; if the new version cannot connect, someone has to reinstall it.
- The console's own rollback has no automatic test.
- **Owner → Updates** shows the history of the machines on the server you are looking at only.

14 — What is new in V2.4

V2.3 of this pack was written on **16 October 2026**. **V2.4** was written on **27 October 2026**, the week before the product goes public on **1 November 2026** — the real website, signups and live card payments. Documents 00–09 have been updated to match; documents 10–13 still list what V2, V2.1, V2.2 and V2.3 added.

Unless it says otherwise below, everything was **built and passed its automatic test tools** but has **not yet been tried on real customer computers**. The hardware checklist is `docs/TEST-3.6.md`, sections **DF to EC** (checks 740–932), and `docs/HANDOVER.md` **§149–§172** is the full engineering record.

The two decisions that changed the business

What	In short	Record
Three plans	**Full** (R500 a month for five people), **the Basic Version for Remote Workers** (R50 a month, one person reaching 3 to 20 of their own computers at R10 each extra, attended or unattended) and **the Free Version** (one computer, unattended with a password, nothing to pay). Free moves up to either; Basic up to Full or down to Free. In the code, Free and Basic share the one-person rules (`FreePlan`), stored as `"free"` and `"basic"`	§161–§163, DT–DV
Five seats; more on request	A company asks on its Billing page; we allow up to a number on **Owner → Approvals**; each seat past five is R300 a month, accepted by the company's owner first, charged through PayFast. **Five paying companies at a time** (Owner → Approvals sets it), others go on a waiting list; a pilot can be **ended by 30 days' notice**	§153, §160, DL, DS

Remote support

What	In short	Record
The Allow prompt comes to the front	Windows was taking the always-on-top mark off it, so it opened behind other windows. It now takes the front and stays there while it counts down	§149, DF
The tray app follows an update	Computers updated themselves (seen on hardware: four releases on one PC), but the tray app never restarted into them — it was reading its own old version number. Fixed; each computer needs one sign-out or restart after the deploy that carries the fix	§152, DI

What	In short	Record
A technician asks for outside help	In a session, **Tools** → **Get outside help…** makes a code for that session, with a message to copy, WhatsApp or email. Whoever types it waits for that technician to let them in; the person at the computer is **told**, not asked	§155, DN

Machines and customers

What	In short	Record
The name at a support request names the machine	An unlinked machine shows the name last typed into Request Support, in grey italic, instead of DESKTOP-ABC123, and linking it offers that person as a contact	§150, DG
The installer asks who the computer is for	Name, surname and email, required in the install window; it names the machine the same way and becomes a contact when the machine is linked	§156, DO
Removing a client	A client added twice or spelt wrong: everything on it (tickets, machines, people, knowledge base, recurring tasks, chats) moves to the right client, then it is deleted. Owner, supervisor or office administrator	§172, EC

Tickets

What	In short	Record
Latest activity	Every update on the tickets as it is posted, newest first; everybody's or only mine; by client or by who wrote it; a count on the tab and in the console	§164, DW
Sort by any column, and due dates	Every heading in the ticket list sorts, in the portal and the console; every ticket can have a due date, red when overdue	§165, DX
Job types	Each company switches off the categories and steps it does not use; a check refuses any choice that would break a route, and a ticket keeps the switches it was opened with	§166–§168, DY

What	In short	Record
@ lists the people on the ticket	Typing @ in a comment box offers the people on the ticket first	DK
Tickets by email: Sign in with Microsoft	A Microsoft 365 mailbox can now be connected by signing in, with no password stored. Needs Clint's one-time app registration (`docs/MICROSOFT-365.md`)	§151, DH

The company and the platform

What	In short	Record
A company takes its data with it	Portal → **Export**: one zip of clients (re-importable), tickets with their stories, photos, files and the knowledge base. We can export a suspended or closed company for its owners	§154, DM
A free account from the owner pages	Owner → Free version makes an account and emails the links, with a technician copied in to help set it up	§157, DP
The outside security review	An outside reviewer (19 October) found no way into another company's data. Fixed: a lockout that a stranger could cause, the holding page's headers, a content-security policy (report-only for now), stored pictures checked whole, and more. Document 03	§158, DQ
Portal addresses use the company's own numbers	`/portal/t/12` is the company's ticket #12, not the database's row number, so an address gives nothing away about other companies. Old addresses keep working	§159, DR
The health check switch says what it does	It read as "install on every client", so it was left off. Reworded, and a save with it off says so in red	DJ

Going public

What	In short	Record
`-RealSite` opens signups and the chat bubble	Every deploy wrote "signups off". Now the holding page and signups move together; with `-RealSite` our site shows a chat bubble saying **The Remote Support Program**, answered by Anywhere IT. **From 1**	§169, DZ

What	In short	Record
	November every deploy needs `-RealSite`, or the holding page comes back	
The website speaks beyond IT companies	It leads with *businesses that look after other people's computers and jobs* — IT companies, software and point-of-sale suppliers, in-house IT teams, repair shops, call-out businesses, remote workers — and uses the words South Africa searches (*job cards*, *remote access*, *live chat*). **The logo** is on every page	§171, EB
Knowing when the site is down	The office PC checks the site every 2 minutes and emails Clint once when it is down and once when it is back, plus the certificate and the backup copy. **Not installed on the office PC yet** (`deploy/monitor/README.md`)	§170, EA
A deploy cuts connections once, not twice	The platform now starts after Caddy restarts, so machines reconnect once per deploy	§165

New test tools

`consentfront`, `requestedname`, `m365check`, `trayversion`, `placescheck`, `exportcheck`, `helpcheck`, `installedfor`, `freeaccount`, `addresscheck`, `basiccheck`, `activitycheck`, `ticketsortcheck`, `jobtypecheck`, `jobtypewire`, `jobtypepage`, `clientremovecheck`, and `Test-SiteWatch.ps1`; with changes to `seatcheck`, `pilotcheck`, `securitycheck`, `freecheck`, `freeinstall`, `webchatcheck`, `sitecheck`, `mentioncheck` and `healthcheckwire`. The ones that use a database run on **both** SQLite and PostgreSQL.

Putting V2.4 live

1. **Signed deploys** (`deploy\Publish-Platform.ps1` with the YubiKey plugged in). The database changes apply by themselves. The console and customer computers update themselves (document 13).
2. **1 November**: the deploy with `-RealSite` (opens the website, signups and the chat bubble), PayFast's **live** account (`deploy\Set-PayfastKeys.ps1`, then `Payfast:Sandbox` false). **Every deploy after that needs `-RealSite`.**
3. **Install the site check on the office PC** (`deploy/monitor/README.md`), and check the office copy of the backups is running.
4. **Sign in with Microsoft** only works once Clint has registered the app (`docs/MICROSOFT-365.md`); its secret runs out — diary it.

Known gaps

- **BUGS #13**: a message a machine sends in the same instant as its hello can be lost. No real program is known to do that. Not fixed.
- The content-security policy only **reports** for now: 45 pages still have script written into them.
- The site check cannot see anything while the office PC is off or the office internet is down.
- Nothing in this list has been tried on real customer computers except the self-updating agents (§152).

15 — What is new in V2.5

V2.4 of this pack was written on **27 October 2026**. **V2.5** was written on **28 October 2026** and is **the last version before the product goes public on 1 November 2026** (the real website, signups and live card payments). Little changed in the code in that day; what changed most is the pack itself.

The pack

What	In short
The guide is in the zip	`The Remote Support Program - Guide.pdf` sits at the top of the zip, beside this pack's start page: the 30-page guide for anybody thinking of using or investing in the product (what it is, remote support, jobs and tickets, the company side, the prices). It is the same file as `docs/brochure/` in the source code, printed from `brochure.html` by `docs\brochure\Print-Guide.ps1`
Documents 00, 02 and 07	Brought up to 28 October

The product

What	In short	Record
The pricing page's country switch	Choosing another country changed only the full version's price; the Basic price, its extra computer and the extra seat still said South Africa's. All four figures now come from the one price list (`Services/Pricing.cs`) for each country	§173, ED (933)
Job types shown to an owner once — really once	The dashboard was meant to send an owner to Job types once. It actually kept sending them there on every Dashboard click until they pressed **Save** or **Keep everything**, so an owner who just looked and clicked away could not reach the dashboard. Now the trip itself is remembered (`Tenant.JobTypesShownUtc` , migration `JobTypesShown`). `jobtypepage` 34 of 34 on both databases (32 with the old rule)	§167

Not yet on the live server: the Job types fix. It is committed and goes out with the next signed `Publish-Platform.ps1` , which applies its migration. After that deploy, an owner already sent to Job types goes there

once more, then never again. (The pricing fix was in the build of 27 October, 17:20, made after it was committed; the page itself stays behind the holding page until 1 November.)